Agents Read More Than They Use

Learning Tool-Output Projections from Execution Traces, and When They Pay Off

Alessio Pomaro
Independent Researcher, ORCID 0009-0008-6975-996X

Preprint, version 2. tracetrim 1.0.0; Studies 1 and 2 ran with versions 0.1 and 0.2, Study 3 with 0.4

Abstract

Language-model agents read tool outputs written for programs: most of what an API returns is never visibly reused, yet is paid for at every turn. We learn from an agent's own traces which fields of each tool output it visibly reuses and remove the rest, protecting identifiers and letting the agent recover any hidden field with an `expand` tool. In live runs of tracetrim 0.4 on `gpt-5.6-terra`, under frozen protocols, a fixed projection with `expand` lowered the median LLM API cost per run by 30% and 24% on two agents with every implemented gate passing, though a stricter audit found billing diagnoses less complete with trimming, so preservation of answer quality is not yet established; when a changed task needed removed fields, adaptive learning kept them from the next run in 16 of 16 cases. Offline, with today's defaults, the learned projection removed 63%, 52% and 65% of held-out tool-output tokens on three of five agents, and 40% to 56% of what a minifier and a static metadata filter fixed in advance leave there. Earlier studies (versions 0.1 and 0.2) explain these defaults: learned from tool calls alone, as a gateway would, the 0.1 aggressive projection missed a used field in 49% of the held-out structured calls it trims on the real-API agents, against 4% from complete traces; replays located three failure mechanisms (transformed values, fields used only to reason, invented identifiers), which the safeguards of 0.2 repaired on the conversations already observed. The method helps most where outputs carry much the task does not use, least where the agent cites most of what it reads; it has not yet been validated on tasks held out from development. The library, a telemetry report and an MCP proxy are released.

Status. Preprint, not peer reviewed. DOI 10.5281/zenodo.24189959.

1 Introduction

An agent built on a large language model works by calling tools and reading what they return. The tools are usually written for programs, not for models: a keyword-research API returns monthly volumes, click-through data, backlink counts and a dozen normalisations of each; a payment API returns every field of a charge, null or not; a code-hosting API embeds a full user object, with fifteen navigation URLs, in every issue. All of it enters the context, is paid for at every subsequent turn, and competes for the model's attention with what the task needs.

Practitioners handle this by hand: a wrapper function per tool that keeps the fields the task uses. This works, but it requires knowing which fields those are, for each tool and each task, and keeping the wrappers in step with the APIs. Generic compressors avoid the per-tool work, but they mostly remove structural redundancy; those that learn per tool learn from what the model asks back, not from what the task's answers use.

This paper asks whether the hand-written selection can be *learned*. The agent's own execution traces show what it did with each tool output: which values reappear in the arguments of later calls and in the final answer. A field whose values never reappear, across runs of the same task, is a candidate for removal. We call the resulting filter a *learned projection*, implement it in an open-source library (`tracetrim`), and test it on five agents: three that use real APIs

(an editorial planner and a content-gap analyst on a search-data API, a literature scanner on a scholarly catalogue) and two that use simulated servers whose payloads follow the public OpenAPI descriptions of a payment API and a code-hosting API.

Since version 0.4, the library protects identifiers, recognises transformed values, writes compact JSON, offers `expand` for any hidden field and can keep learning while the agent works. Measured with these defaults, in live runs on `gpt-5.6-terra`, a fixed projection with `expand` lowered the median LLM API cost per run by 30% and 24% on two agents with every implemented gate passing, and when a changed task needed fields the projection had removed, the agent recovered them and they were kept from the next run in 16 of 16 cases. Offline, the projection learned with these defaults removed 40% to 56% of what a minifier and a static metadata filter leave on three of the five agents. The earlier studies, run with versions 0.1 and 0.2, are where those defaults come from. **Study 1**, with a protocol frozen before any run, measures how much a projection learned from other runs removes on a held-out run, where it must be learned to work, whether it transfers between tasks, and whether the agent's answer survives when the conversation is replayed with projected tool outputs. It finds large savings and three precise ways in which the projection fails: values the agent *transforms* before writing them (a Unix timestamp written as a date), fields the agent uses only *to reason*, and, most seriously, an agent that *invents identifiers* when the identifier field has been removed. **Study 2** adds two safeguards motivated by those failures and tests, again with a protocol frozen in advance, whether they remove those specific losses, on the conversations that showed them, while keeping most of the saving. **Study 3**, added in version 2 of this paper, measures in live runs, under two further frozen protocols, what version 0.4 of the library added: a *fixed* projection with `expand` (E1), learning that *adapts* while the agent works, with `expand` requests as evidence (E2), and, without trimming, a text that teaches the agent how to organise its tool calls, on a second model (E3, reported in an appendix). It also reports offline analyses, among them a comparison with a minifier and with a static filter fixed before any number, and post-hoc audits of the answers, each labelled as such.

The contributions are: (i) a method to learn tool-output projections from traces, with a leave-one-out evaluation that estimates savings and misses on runs the projection has never seen; (ii) evidence that, in these agents, learning without the final answer misses far more of what is used (with the aggressive projection of version 0.1, 49% against 4% of the held-out structured calls it trims on the three agents with real APIs), so a proxy placed only between agent and tools (an MCP gateway) learns poorly by itself; (iii) a characterisation of the failure modes, including identifier fabrication; (iv) safeguards and their test; (v) an open-source implementation with a bilingual telemetry report that is useful on its own, independently of any trimming; (vi) a live measure of that implementation's fixed trimming and adaptive learning, and of a call-organising text, with the hypotheses that failed reported as failed; (vii) an offline comparison with minification and a pre-registered static filter, which separates what learning adds from what any serialiser or metadata filter removes.

2 Related work

Prompt and context compression. A large body of work compresses what a language model reads. Token-level methods drop tokens scored as uninformative, by self-information [9] or by a small language model [5], possibly conditioned on the question [6]; LLMLingua-2 learns a task-agnostic compressor [17]. RECOMP compresses retrieved documents for retrieval-augmented generation [24], and gist tokens learn soft prompt representations [15]. These methods operate on text and decide token by token; the projection studied here operates on structured outputs and decides field by field, and its output is still readable, diffable JSON. The motivation is shared: models use information in long contexts unevenly [11], so what is removed is not only cost.

Approach	Unit removed	Signal	Training or extra inference	Recoverable	Live evaluation
LLMLingua family [5, 17]	tokens of text	small-model scores	extra inference	no	– (not agents)
ACON [7]	observations, history	guidelines optimised from failures	LLM calls; distillation	no[a]	success, tokens[a]
Observation masking [10]	older observations	recency	none	no	solve rate, cost
Squeez [8]	lines of one tool output	the current query	fine-tuned small model	no[a]	–[a]
Policies study [19]	history, tool results	recency, content type, summaries	none, or summariser calls	no	success, tokens, latency, cost
Coca-Cola [12]	observations (to latent tokens)	next-action matching	trained compressor, open weights	no	success, tokens, runtime
Headroom [4]	by content type	rules, per-tool usage	none for its JSON path[a]	yes	–[b]
tracetrim (this work)	fields of structured outputs	reuse in the agent's own text	none	yes (`expand`)	gates, tokens, cost

Table 1: The closest approaches, on the dimensions that separate them, as their sources describe them. "Live evaluation": what the source measures in agent runs. [a]As far as the source states. [b]A tool, not a study; we compared its token counts on our replays (Section 6). tracetrim has not yet been validated on tasks held out from its development (Section 9).

Tool use and the agent loop. Agents that interleave reasoning and tool calls [25], learn when to call APIs [20] or operate over thousands of real APIs [18] accumulate tool observations at every turn. The Model Context Protocol [14] has made connecting third-party tools routine, which moves the choice of what an agent reads from the agent's author to the tool's.

Compressing what agents observe. Closest to this work are methods that reduce agent observations. ACON optimises, in natural language, compression guidelines for observations and history from the agent's failures, and distils the compressor into smaller models [7]. For software-engineering agents, masking older observations halves cost relative to the unmanaged agent while matching the solve rate of LLM summarisation [10]. Squeez fine-tunes a small model to return, for a focused query and one tool output, the smallest verbatim evidence block the agent should inspect next [8]. A systematic study of context-compression policies for agents (truncation, clearing of tool results, summarisation and their combinations, about 35,000 runs of open-weight models on two software-engineering benchmarks) finds that fewer tokens need not mean faster or cheaper runs, partly because of cached-prefix pricing, and that no single policy is best across task success and cost [19]. Coca-Cola compresses tool observations into continuous latent tokens, with a compressor trained so that the agent's next actions match those it takes with the full observations; it needs gradients through the agent and was evaluated on open-weight models in web-search question answering [12]. Our setting differs in three ways: the projection is learned from the agent's own traces without training or calling any model, so it adds no inference cost, works with models reachable only through an API, and its decisions are a reviewable list of fields with a way back; it targets structured outputs of arbitrary APIs rather than code-agent observations or free text; and we measure where such learning fails, not only how much it saves. Table 1 compares these approaches on the dimensions that separate them. It is a map of the closest work we found, not a systematic review.

Tools. Headroom compresses agent context in front of the model, routing content to compressors by type [4]. Two of its components are close to ours: one learns, over repeated calls of a tool,

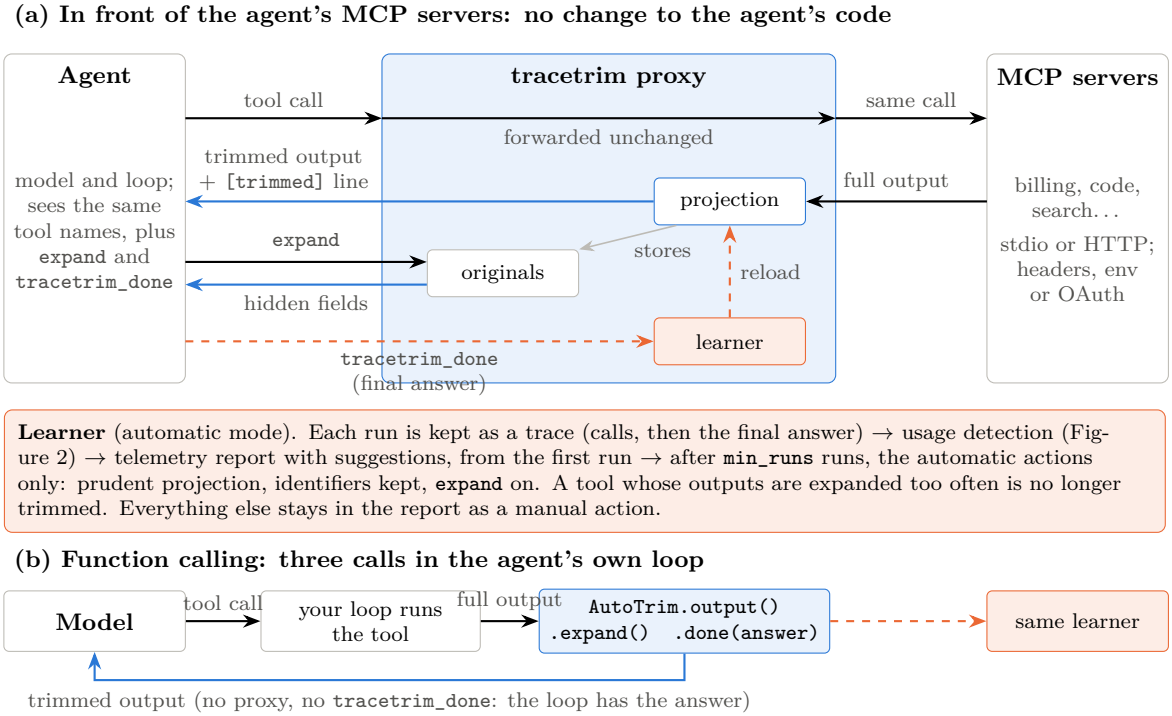


Figure 1: Where tracetrim acts. (a) As an MCP proxy, it stands in for the agent's servers: calls are forwarded unchanged, only results are rewritten on their way back, and a trimmed result ends with a line listing what was hidden, which `expand` returns from the stored original without calling the server again. The agent passes its final answer to `tracetrim_done`, which closes a run for the learner. (b) With function calling the same learner sits in the agent's own loop. The automatic mode (dashed) is part of version 0.3 and was not evaluated in Studies 1 and 2, which used offline learning and a fixed projection; Study 3 measures it through (b).

which items are retrieved and which fields carry signal, and feeds this into how JSON arrays are reduced; another keeps the originals and lets the model retrieve them with a tool, as our `expand` does. The differences are the signal and the unit of decision: we learn from what the agent writes, final answer included, separately for each task, and the decision is a list of fields a person can review. In our baseline each conversation was compressed once, so its per-tool learning had no history. TOON is a compact, table-oriented serialisation of JSON for language models [21]. Gateways for the Model Context Protocol reduce context in other places: mcp-compressor mainly compresses the descriptions and schemas of the tools exposed, with an opt-in bound on response size whose originals stay recoverable [2], and Sift stores full payloads outside the context and lets the agent query them [13], which is close in spirit to our `expand`.

Fabricated identifiers. Language models fabricate plausible identifiers when the real ones are not in front of them: bibliographic references [23] and software package names [22]. We observe the same behaviour inside an agent loop, caused by the removal of an identifier field, which makes identifiers a category a projection must never remove.

3 Method

Figure 1 shows where the library acts and how it learns; Figure 2 shows how it decides which fields an agent uses. The rest of this section describes each step.

Traces. A trace is the ordered record of what the agent was given and what it wrote: prompts, tool calls (the arguments the agent wrote and the raw output it received) and messages, the

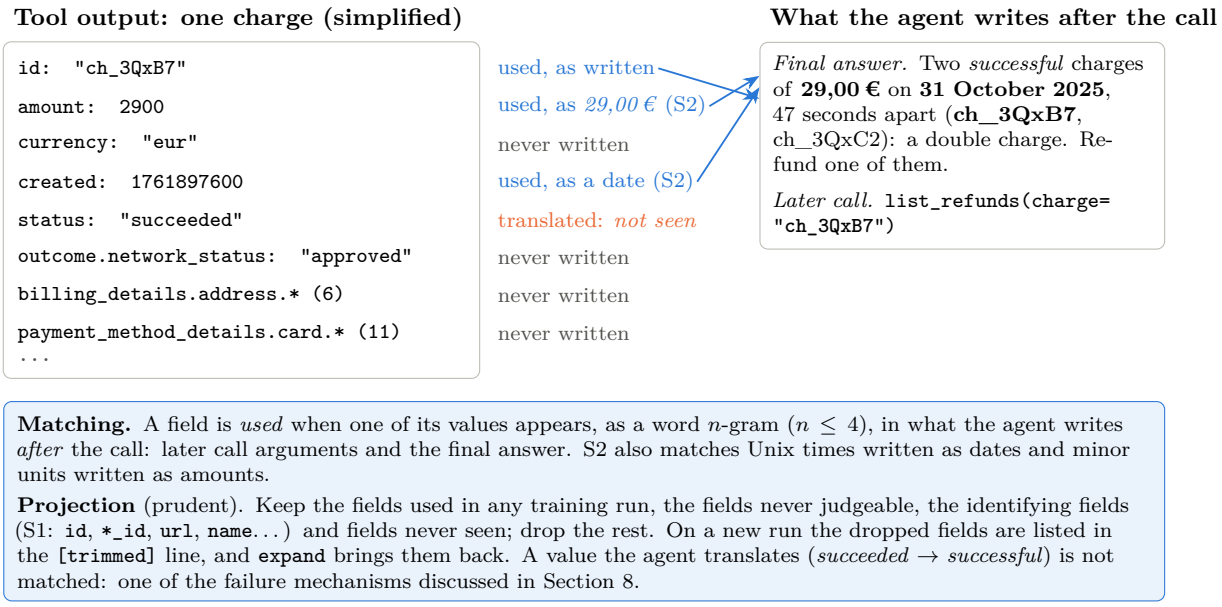


Figure 2: How usage is detected, on an illustrative billing call (values simplified). Blue: values matched in what the agent wrote after the call, directly or through the safeguard S2; orange: a value the agent used after translating it, which matching cannot see.

final answer included. The library reads a neutral JSONL format and the Python exports of the OpenAI Playground.

Usage. Each structured output is flattened into leaves identified by a path in which list indices are collapsed (`items[].keyword_info.search_volume`). A leaf is *used* when one of its values appears in something the agent wrote *after* receiving it and not before. Values are matched as word n-grams ($n \leq 4$): short strings as a whole, prose by any 4-gram, numbers in their plain form and with dot thousands separators, as Italian writes them (the library matches comma separators too since version 0.3.4; the studies did not). Values too generic to attribute (booleans, integers below 100, one-word strings shorter than four characters) are not *judgeable*. A value that already appears in the call's own arguments is an *echo*: not a use, but the record's key (the keyword next to its volume), and it is always kept.

Projection. For each tool, the *aggressive* projection keeps the used fields and the echoes; the *prudent* one also keeps every field that was never judgeable. Fields never seen while learning pass through, so that a field added to an API is not hidden silently. Projected outputs are re-serialised without ASCII escapes.

Leave-one-out evaluation. For each trace, the projection is learned from the other traces of the same task and applied to it. Two quantities are reported: the tool-output tokens left (counted with the `o200k_base` tokenizer, each output once), and the *misses*: calls in which the projection removes a field that this trace used later. Usage is *detected textual reuse*, neither a lower nor an upper bound of what the agent uses: a field read only while reasoning leaves no trace in the text (reasoning is encrypted in the APIs used here), and a value that coincides with another field's value, or a field kept for one element of a list, is counted as used where it was not. "After" means a later model turn: calls emitted together in one turn are generated before any of their outputs exist (deviation 14).

Learning source. Usage can be learned from everything the agent wrote (*full*) or only from the arguments of later tool calls (*calls*), which is what a proxy between the agent and its tools can observe.

Structural compression. After projection, outputs can be cleaned (null and empty values removed) and written in a TOON-style encoding [21] implemented in the library (not the reference implementation): indented `key: value` lines, with arrays of uniform flat objects as tables whose field names appear once.

`expand`. At run time, a projected output is followed by a line listing the hidden fields, grouped two levels deep and largest groups first, and the agent can retrieve any of them with an `expand` tool. This turns a loss into a round trip, provided the agent knows what is missing.

Safeguards (version 0.2, Study 2). S1 keeps identifying fields (`id`, `*_id`, `doi`, `url`, `number`, `title`, `name` and similar) whatever the usage says. S2 recognises transformed values: Unix timestamps and ISO dates also match as written dates, in Italian and English; integers in money-named fields also match as major units with two decimals; URLs also match by their last segment.

3.1 Detected reuse, required information and misses

Three terms are used throughout with a fixed meaning. *Detected reuse*: a leaf path of a tool output is detected as reused in a run when one of its judgeable values matches, by the rules above (and, with S2, the transformations it recognises), something the agent wrote in a later model turn and not before. It is a property of the trace and of the matcher. *Required information*: for a task and a run, the leaf paths whose values the agent needs to produce a correct answer, whether it writes them as they are, writes them transformed, or only reads them to decide. It is defined by the task and its correct answers, not by the trace, and in general it cannot be observed. *Projection miss*: a held-out call in which the projection removes a path that the held-out run reused, as detected. Misses are counted against detected reuse, not against required information: a projection without misses has removed nothing the detector saw used, which is not the same as removing nothing required.

Why absence of detected reuse certifies nothing by itself. A tool returns two issues, each with an identifier and a title: issue A, titled "staging deploy fails", and issue B, titled "flaky unit test". In a first task the agent must report the open issue with the earliest identifier; in a second, the issue that reports a failed staging deployment; in both it answers with an identifier only. Issue A is both the earliest and the one about staging, so the two tasks, though their instructions differ, can produce the same sequence of calls, outputs and answer, and therefore the same detected-reuse signal, in every run. In neither is the title ever detected as reused, since its value never appears in what the agent writes. A learner that relies on that signal cannot tell the two tasks apart: if it removes the title in the first, where it is not needed, it removes it in the second, where the agent needs it to choose. Study 1 met this case in CI triage (Section 5). However many traces are collected, the absence of detected reuse supports removing a field only under an added assumption, for instance that every required value is written verbatim, which fields used only to reason violate. This is why a learned projection is treated here as a proposal, with identifiers protected, the hidden fields listed and `expand` as a way back.

4 Experimental design

Studies 1 and 2 follow protocols frozen before their runs and tagged in the repository (`protocol-v1`, amended as `protocol-v2`; Study 2 as `protocol-v3`). Every change made after freezing is listed, dated and explained in the protocol's "Deviations" section and summarised in Appendix A.

Cases. Five agents, all with `gpt-5.6-terra` at default parameters, in a loop of our own (OpenAI Responses API with function calling), because hosted tool integrations do not expose tool outputs to the client.

- **C1, editorial plan**: twelve tools of a search-data API (keyword research, trends, search results, rankings, page parsing) reached through its MCP server; the agent plans articles for the author's own website.
- **C2, content gaps**: search results and page parsing; the agent compares what ranks on Google with what the website has published.
- **C3, literature scan**: two tools on the OpenAlex catalogue (CC0), five results per search.
- **C4, billing support**†: seven tools of a simulated MCP server whose payloads follow the public OpenAPI description of the Stripe API; the agent answers five customer tickets with known correct answers.
- **C5, CI triage**†: six tools of a simulated MCP server following the GitHub REST API description; the agent explains the week's CI failures on `main` and links them to issues and pull requests, again with known correct answers.

†Simulated payloads are generated from the OpenAPI schemas with a seeded generator; because we design them, C4 and C5 are reported separately and never pooled with the real-API cases. Inputs name only the author's own domain; a gate in the tool wrapper rejects any other domain for page parsing, ranking targets and `site:` queries.

Study 1. *Phase A*: eight live runs per case, used for usage, leave-one-out projections, learning source and transfer between C1 and C2 (on the tools they share). *Phase B*: the third run of each case, chosen before looking at the data, is replayed up to the final answer, changing only the tool messages: raw, prudent, aggressive, and prudent plus structural compression; two runs per arm; projections learned from the other seven runs. Headroom (`headroom-ai` 0.39.1) is applied to the same conversations as an illustrative comparison of tokens only, in the configuration chosen below, without any evaluation of the answers it would lead to. It ran *stateless*: each conversation compressed once, so its per-tool learning had no history of these tools, while the projection learned from seven runs. It was configured to compress everything, with no protection of recent messages and a target ratio of 0.2 (an aggressive setting, not one tuned for these tasks); its input was the conversation's user message, tool calls and tool messages, without the system prompt, with the model parameter set to `gpt-4o`. *Phase C*: live runs on C1 and C4 with the prudent projection and `expand`, against raw runs, three per arm.

Study 2. The Phase B conversations are replayed again with the safeguards on (arm `v02`, and `v02` plus structural compression) against the raw arm, five runs per arm.

Measures. *Gates* are binary: the answer parses, has the required structure, and the facts it attributes to tools match the tool outputs stored in the trace. As implemented they are narrower than the protocol's wording: C1 checks each reported volume against the volumes the run's outputs give for that keyword, and skips a keyword that has none; C2 checks that each cited URL was returned by the search tool, not its title; C3 checks each work's title through its identifier, and that a declared identifier exists; C4 checks that every object identifier cited exists; C5 checks run, job and issue numbers. Appendix B measures what this leaves unchecked. In Phases A and B and in Study 2 the stored outputs are the raw ones, whatever the agent was shown; in Phase C, where the live loop stores what the agent received, they are the projected outputs and what `expand` returned (Section 5.5). *Thermometers* are graded: editorial metrics for C1, coverage metrics for C2 and C3, and, for C4 and C5, keyword tests for the diagnoses and links

Case	Runs	Calls	Tool-output tokens	Used	Re-serialised	Prudent	Aggressive	Misses	Misses (calls only)
Editorial plan	8	173	817,865	32%	−13%	−63%	−68%	2/166	69/166
Content gaps	8	136	527,542	84%	−4%	−12%	−16%	9/90	26/90
Literature scan	8	61	2,262,885	19%	0%	−70%	−81%	3/61	61/61
Billing support†	8	84	154,833	17%	−17%	−69%	−84%	9/84	61/84
CI triage†	8	104	599,808	18%	−7%	−77%	−82%	3/64	64/64

Table 2: Phase A, eight live runs per case. *Used*: share of tool-output tokens left by the in-sample aggressive projection (used fields and record keys): detected reuse, not an exact measure of use. *Re-serialised*: change from re-serialising each JSON output compactly, with no field removed (the lossless part of the next two columns). *Prudent*, *Aggressive*: change in tool-output tokens on held-out runs. *Misses*: held-out calls in which the aggressive projection removes a field the run used later, learning from everything the agent wrote, or only from tool calls. †Simulated tools.

whose correct answers are known. Results are reported as medians with ranges (Appendix B); with two to five runs per arm no significance test is claimed. The pre-registered rule for quality (H2) is relative: an arm *meets* it when it passes every gate in at least as many runs as the raw arm and its median on every thermometer lies within the raw arm's range. It says nothing about absolute quality: where no raw run passes the gates, an arm meets it with none passing either.

5 Study 1: what a learned projection removes, and what it breaks

5.1 How much the agent uses, and how much a projection removes

Table 2 summarises Phase A. Only a minority of the tool-output tokens carry fields the agent visibly used (with the record keys): 32% in the editorial plan, 19% in the literature scan, 17% and 18% in the two simulated cases. On runs it had not seen, the prudent projection removed 63%, 70%, 69% and 77% of the tool-output tokens of these cases; in the editorial plan this is above the 30% threshold of H1, which holds. Part of each figure involves no removal at all: the tools return indented JSON, and writing the same payload compactly and without escapes already removes 13% in the editorial plan, 17% in billing support and 7% in CI triage (column *Re-serialised*), and almost nothing in the literature scan, whose outputs were already compact. The content-gap analyst is the boundary case: it cites most of what search results contain (URLs, titles, snippets) and its page parses are prose, which a field projection cannot touch; its projection removed 12%. Without the two editorial-plan runs in which a tool call failed (deviation 6), the saving there is 63% and the misses 3/124.

5.2 The learner needs the answer

Learning only from later tool calls, which is all a proxy between agent and tools can observe, multiplied the misses of the aggressive projection of version 0.1 (Table 2): held-out structured calls that the projection trims and in which a used field would have been removed went from 14/317 to 156/317 over the three cases with real APIs, and from 12/148 to 125/148 over the two simulated ones; from 2/166 to 69/166 in the editorial plan, and to every trimmed call in the literature scan (61/61) and in CI triage (64/64): those agents use what tools return only in the final answer. H3 (more than three times as many) holds in four cases and fails in the content-gap analysis, by a narrow margin (9/90 against 26/90, a ratio of 2.9). In these cases a projection can be *applied* anywhere, a proxy included, but learning it without the answer misses far more.

5.3 Projections are task-specific

Learned on the editorial plan and applied to the content-gap analyst, on the one tool they share, the projection missed a used field in 67/90 calls, against 9/90 for the analyst's own projection

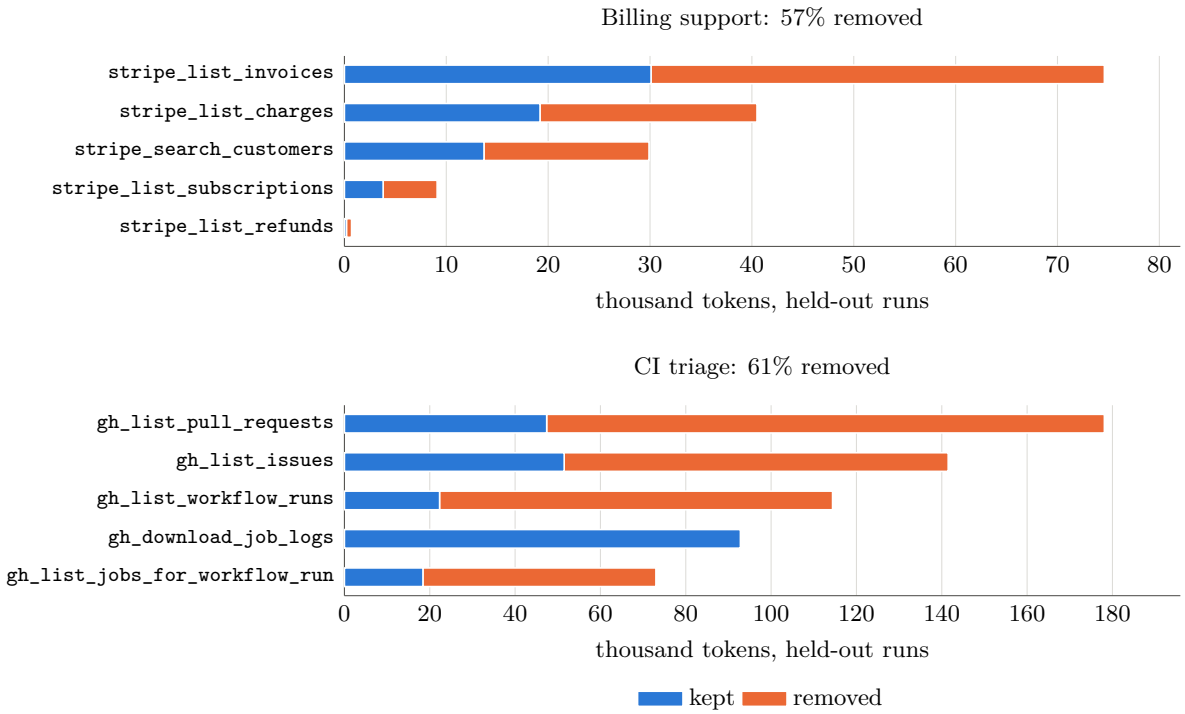


Figure 3: Where the tokens go, per tool, in the two cases with simulated tools (released in full): tool-output tokens on held-out runs, split into what the prudent projection keeps and what it removes. The projection here has the 0.2 safeguards on (Section 6), as the library now does by default, so it removes less than the 0.1 projection of Table 2. This is the breakdown the telemetry report shows for every agent; generated from the traces by `experiments/paper_figures.py`.

(H4). What an agent needs from a tool depends on the task, not only on the tool.

5.4 What changes in the answers (Phase B)

Replaying the same conversation with projected tool outputs (Table 3) removed 59%, 68%, 69% and 77% of the tool-message tokens with the prudent projection, against 28%, 17%, 33% and 31% for Headroom on the same conversations: an illustrative comparison of tokens in the configurations chosen (Section 4), not of methods, since the quality of Headroom's outputs was not evaluated. H2, as frozen, was rejected in almost every case: with two runs per arm the raw arm's range is often a single value, so any difference counts, including improvements. Reading the answers rather than the rule, the losses are concentrated in three mechanisms, each traced to a specific field.

Transformed values. In billing support, the end of the subscription period arrives as a Unix timestamp and the agent writes it as a date. Usage detection does not recognise the rewritten value, so the field is judged unused and removed; every projected arm then answered the customer who asked until when the service would be available with a vague sentence instead of the date.

Fields used only to reason. In CI triage the projection removed issue titles: the agent reads them to decide which issue a failure belongs to, but writes only the issue number. Without titles, every projected arm failed to link the staging-deployment failure to the issue that reports it.

Invented identifiers. In the literature scan, the work identifier arrives as a URL and the agent writes its last segment, so the identifier field was removed. Asked to report it, the agent did not write `null` as instructed: it wrote sequential identifiers starting from the example in the system

Case	Arm	Tool tokens	Gates passed	H2 (implemented)	H2 (protocol, computable)
Editorial plan	raw	114,712	1/2		
	prudent	−59%	1/2	fails	fails
	aggressive	−64%	2/2	fails	fails
	structural	−60%	1/2	fails	fails
	Headroom (tokens only)	−28%	–	–	–
Content gaps	raw	50,488	0/2		
	prudent	−12%	0/2	fails	fails
	aggressive	−16%	0/2	fails	fails
	structural	−13%	0/2	meets	meets
	Headroom (tokens only)	−10%	–	–	–
Literature scan	raw	227,433	2/2		
	prudent	−68%	0/2	fails	fails
	aggressive	−79%	0/2	fails	fails
	structural	−72%	1/2	fails	fails
	Headroom (tokens only)	−17%	–	–	–
Billing support†	raw	16,931	2/2		
	prudent	−69%	2/2	fails*	fails
	aggressive	−84%	2/2	fails*	fails
	structural	−73%	2/2	fails*	fails
	Headroom (tokens only)	−33%	–	–	–
CI triage†	raw	74,976	2/2		
	prudent	−77%	2/2	fails	fails
	aggressive	−82%	2/2	fails	fails
	structural	−78%	2/2	fails	fails
	Headroom (tokens only)	−31%	–	–	–

Table 3: Phase B: replay of one run per case, only tool messages changed, two runs per arm. *Tool tokens*: raw size, then change per arm. *Gates*: runs in which every gate passed. *H2*: the relative rule (gates passed in at least as many runs as raw, every thermometer median within the raw range), on the thermometers as implemented and on the computable part of the protocol's list, without its "topics covered" (deviation 15); *meets* says nothing about absolute quality (content gaps: no raw run passes the gates). *Also outside the band on the strict test added after the fact (deviation 11).

prompt. The DOIs it reported were correct; the identifiers did not exist. With the full outputs, in all eight live runs, every identifier was real. Removing an identifier is therefore not only a loss of information: it can cause the agent to fabricate one, the failure mode known for references and package names [22, 23].

5.5 Live loop with `expand` (Phase C)

In live runs with the prudent projection and `expand` (Table 4, Figure 4), billed input tokens fell by 56% in the editorial plan and 32% in billing support, and uncached input tokens by 47% and 54%. With the model's list prices at the time (2.00, 0.20 and 12.00 USD per million input, cached and output tokens; no request reached the long-context tier), the cost per run fell by 40% and 25%. Without the two editorial-plan runs in which a search call failed after the retries (one per arm), billed input fell by 50%.

Every run passed its gates. In the editorial plan the gate checks each reported volume against the volumes in the run's own tool outputs: it verified 60, 59, 60 of the 60 volumes of the three prudent runs, and all 60 of every raw run. It skips a keyword for which the run's outputs give no volume, and one such pair was skipped: a volume of 0 for a keyword that no tool output of that run gave a volume for, and that has 40 in the other runs of the same day. Checked after the fact against those runs, 179 of the 180 prudent volumes are confirmed and one is not: a value written without a source, which a gate of this design cannot catch (deviation 13).

The agents did use `expand`: a median of 1 call per run in the editorial plan and 6 in billing support, where the agent recovered the hidden period end in two of three runs. In the third it did not, and passed the keyword test on 3 tickets of five, fewer than any raw run (at least 4); under

Case	Arm	Input tokens	Uncached	Turns	Expand calls	Gates
Editorial plan	raw	847,304	107,274	10	0	3/3
	prudent	374,748	57,117	10	1	3/3
Billing support†	raw	47,202	22,097	4	0	3/3
	prudent	32,161	10,239	5	6	3/3

Table 4: Phase C: live runs, raw tools against the prudent projection with `expand`; medians of three runs. Input tokens are billed tokens from the API, cache included.

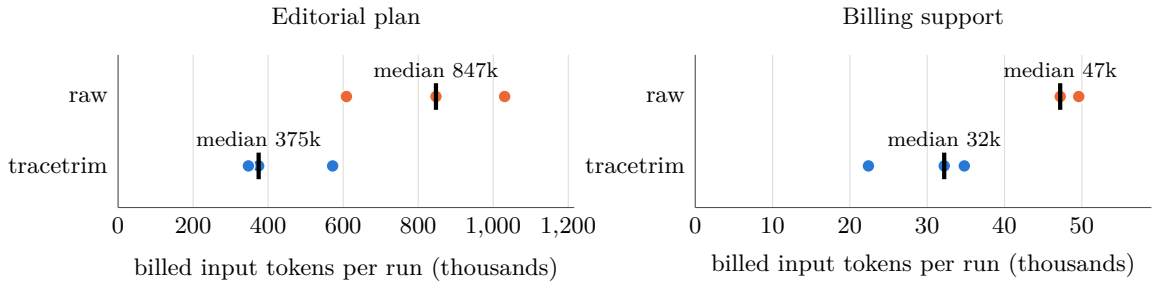


Figure 4: Phase C, every run: billed input tokens per run with raw tools (orange) and with the prudent projection and `expand` (blue); the bar marks the median. Three runs per arm: the spread is part of the result.

the stricter rules both arms pass 12/15 and 12/15 (Table 12). Run-to-run variation in these tasks is large compared with the effects measured.

A limit of this phase: the live loop stored what the agent received, so for the prudent arm the raw outputs of the run itself are not available. The re-check against the other runs of the day can show a discrepancy, not prove what the agent was given; a harness should store the original output, what was shown, and what `expand` returned, separately.

6 Study 2: safeguards

Study 2 replayed the Phase B conversations again, 5 runs per arm, with the projection learned with both safeguards (arm `v02`) and with the same projection followed by structural compression (Table 5). It is a pre-registered check that the safeguards repair the specific failures that motivated them, on the conversations already observed, not an independent validation on new tasks, runs or seeds.

Pre-registered hypotheses. All four held as frozen. *H5*: in billing support the `v02` arm passed the frozen keyword test on all five tickets in 5/5 runs, the period end included, where the 0.1 projection had lost it. The keyword tests accept a negated or contradictory sentence. Stricter rules, added afterwards (deviation 15, Table 12), read only the diagnosis and require the scenario's exact amounts, the year and the outcome, rejecting negations: 20/25 `v02` diagnoses pass them against 23/25 raw ones, every `v02` failure being the decline reason discussed below. These are still rules, not a reading of the answers; the difference is not a statistical result (five tickets per run are not independent observations), but it shows that the hypotheses held as frozen do not by themselves mean that quality was preserved. *H6*: in the literature scan every `v02` run passed the gates (5/5), so no identifier was invented; with 0.1 the prudent arm had passed in 0/2 runs. *H7*: in CI triage every `v02` run found all four expected links, including the one lost in Phase B. *H8* is a token count on fixed replays, so it is deterministic and equals the offline estimate given in the protocol: the `v02` arm still removed 53%, 46%, 57% and 61% of the tool-message tokens in the editorial plan, the literature scan, billing support and CI triage, against 28%, 17%, 33% and 31% for Headroom (tokens only, in the configuration chosen; an illustration, not a comparison

Case	Arm	Tool tokens	Gates passed	H2 (implemented)	H2 (protocol, computable)
Editorial plan	raw	114,712	5/5		
	v02	−53%	5/5	fails	fails
	v02-structural	−55%	2/5	fails	fails
	Headroom (tokens only)	−28%	–	–	–
Content gaps	raw	50,488	2/5		
	v02	−9%	1/5	fails	fails
	v02-structural	−10%	0/5	fails	fails
	Headroom (tokens only)	−10%	–	–	–
Literature scan	raw	227,433	5/5		
	v02	−46%	5/5	meets	meets
	v02-structural	−52%	5/5	meets	meets
	Headroom (tokens only)	−17%	–	–	–
Billing support†	raw	16,931	5/5		
	v02	−57%	5/5	meets	meets
	v02-structural	−62%	5/5	fails	fails
	Headroom (tokens only)	−33%	–	–	–
CI triage†	raw	74,976	5/5		
	v02	−61%	5/5	meets	meets
	v02-structural	−63%	5/5	meets	meets
	Headroom (tokens only)	−31%	–	–	–

Table 5: Study 2: replay of the same runs as Phase B, 5 runs per arm, safeguards S1 and S2 on. Columns as in Table 3. No cell carries that table's asterisk, but the same caveat applies: in billing support the `v02` arm meets H2 under the frozen keyword test, not under the strict test added after the fact (deviation 11; see the text).

of methods) and 24%, 16%, 33% and 18% for a full minifier that removes no field (Figure 5). Measured against JSON already minified, the `v02` projection still removes 39%, 35%, 36% and 53% of the tokens in these four cases (Table 13). The safeguards cost part of the saving (compare 68% with 46% in the literature scan) and bought back the three lost pieces of information.

Outside the hypotheses. In the editorial plan the `v02` arm does not hold under the band rule (Table 5): its gates passed in every run, but the median search volume of the keywords it chose was 1300, outside the raw arm's range (355–590), and on the computable part of the protocol's list so was the mean (2,970 against 2,464–2,874). We read no direction into it: the same projection followed by structural compression stayed in range (590), and in Phase B the prudent arm was in range (590) while the aggressive and structural arms were not (1300 and 1300). With five runs per arm, search-volume measures outside the raw range are a change to watch, not an established effect.

A fourth mechanism, found after the fact. Reading the billing-support answers, we found that the frozen test for one ticket was too lenient: it accepted the diagnosis when "fondi" (funds) or the stem "insufficien" appeared anywhere in the ticket, so advice such as "check that you have sufficient funds" passed it even when the diagnosis did not give the reason for the declined payment. With a strict test on the diagnosis alone (added after the data were seen, and reported beside the frozen result rather than instead of it), the raw arm gave the reason in 3/5 runs and the `v02` arm in 0/5. The reason arrives as an English sentence and the agent writes it in Italian: a *translated* value, which neither usage detection nor safeguard S2 recognises. The `v02` answers were not wrong ("the card was declined") but less informative.

Cleaning and format change behaviour. The structural arm adds two things to the `v02` projection: cleaning, which removes nulls and empty values (inside lists too, which shifts the positions of the remaining elements), and the TOON-style encoding. Together they changed answers. In the editorial plan the gates passed in 2/5 runs instead of 5/5, each time on the same

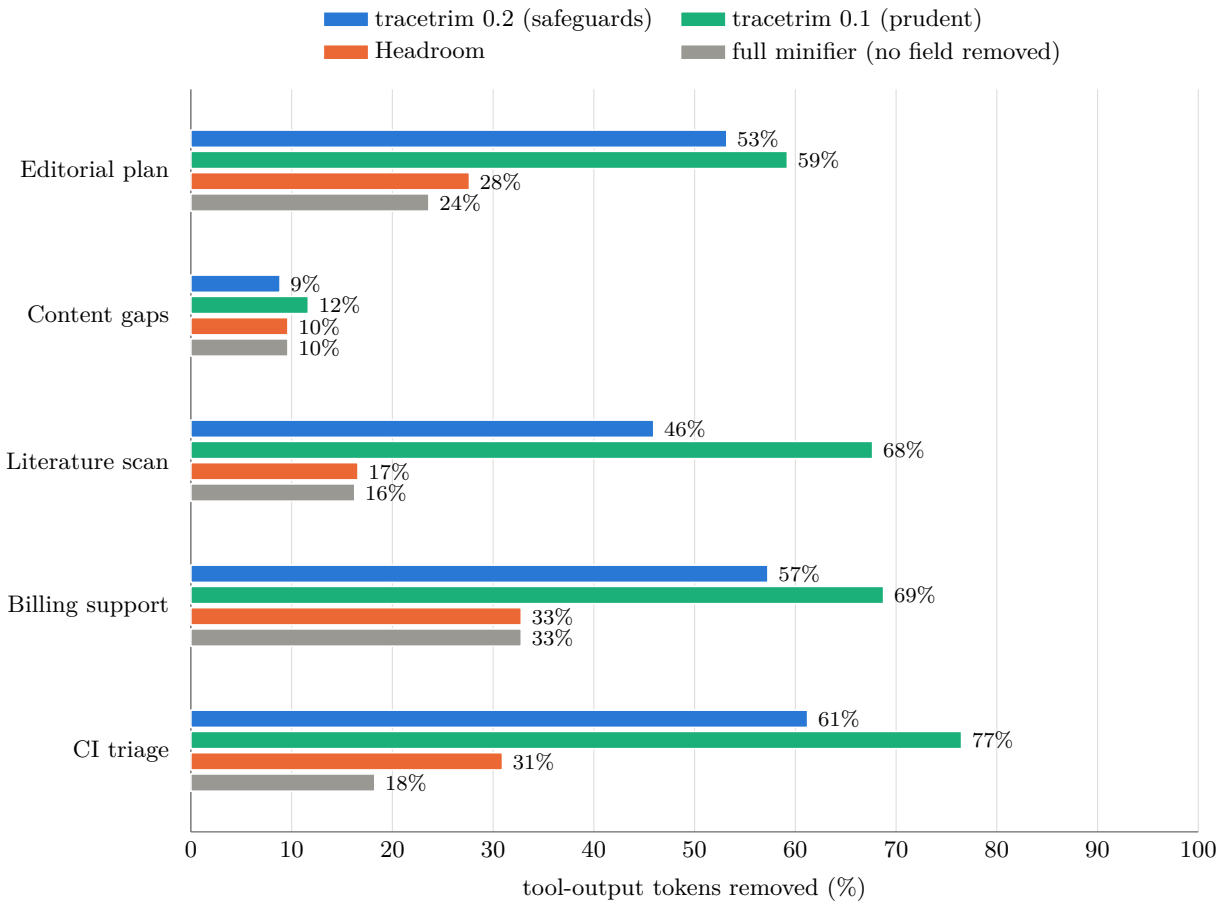


Figure 5: Tool-output tokens removed on the same conversations, per case: the learned projection without safeguards (0.1, Phase B), with them (0.2, Study 2), Headroom 0.39.1 configured to compress everything and run once per conversation, and a full minifier, which removes no field (tracetrim's own output is compact JSON with default separators; Table 13 separates the two). Token counts only: the effect on answers is in Tables 3 and 5.

keyword: the search-volume tool, which the prompt names as the source of all volumes, returned that keyword without a volume, while another tool gave one. With JSON the agent used the other tool's figure, against the prompt's instruction to take volumes from the search-volume tool (the gate checks the number, not its source); with the cleaned and encoded outputs it wrote zero. In billing support the arm had all tickets right, under the frozen test, in 2/5 runs. The two changes cannot be told apart here (an arm for each would be needed): what the data show is that the combined treatment shifted how the model resolved an ambiguity. The evidence is thin (one keyword, five runs), and the encoder is the library's own minimal TOON-style implementation, not the reference one [21]; but it is a reminder that a format the model reads less often is not a neutral choice, and its saving has to be validated like any other change to what the agent reads.

The content-gap case. The content-gap analyst failed the gates in most runs of every arm, the raw one included (2/5): when the search API returns a redirect link instead of a result's URL, the agent reconstructs the URL from the breadcrumb, against the instructions. The safeguards neither caused nor fixed this, and the projection saves little on this task (9%): less than a full minifier (10%), which keeps every field. Where an agent cites most of what it receives, minifying is the better choice.

7 Study 3: fixed trimming (E1), adaptive learning (E2) and a strategy text without trimming (E3), with tracetrim 0.4

Studies 1 and 2 learned a projection offline and applied it fixed. Version 0.4 of the library added what this study measures. In *automatic mode* the library learns while the agent works: runs are recorded in a work directory, a run counts for learning once the agent's final answer is known (through `tracetrim_done` behind the MCP proxy, or directly in a function-calling loop), and trimming starts once enough runs are recorded. Each `expand` request is kept as evidence, with two optional corrections: *promotion*, which keeps a field that the evidence shows to be needed, and *sampling*, which passes a share of the results that would be trimmed whole. A tool whose trimmed results are expanded often enough stops being trimmed (*switch-off*). Version 0.4 also ships a *skill*, a text for the agent: its base level explains `tracetrim_done` and `expand`; an optional *strategy* level teaches how to organise tool calls (independent calls together, grouped inputs, when to stop) and prescribes no parameter value. Study 3 changes nothing in Studies 1 and 2.

The three experiments answer different questions. E1 applies a *fixed* projection, learned once, with `expand`; E2 lets the projection *adapt* from run to run; E3 tests the strategy text *without any trimming* on a second model. This section reports trimming and adaptation: E1's trimming arm and its quality, E2, and the offline analyses. The strategy text (E1's strategy arms and E3) and the hosts checked are in Appendix C, summarised in Section 7.5.

Design. Two protocols, each frozen before its first paid run. `protocol-v04.md` (frozen on 11 November 2026) covers E1 and E2: 100 runs of `gpt-5.6-terra`, the model of Studies 1 and 2, at default parameters. `protocol-v04-e3.md` (frozen on 12 November 2026) covers E3, on a second model, `gemini-3.8-flash`: 38 runs after four pilot runs that enter no result. All runs used the library's function-calling adapter in the study's own loop; the MCP proxy was not exercised (that the two adapters take the same decisions is covered by a test, not by these protocols). The protocols list six and one deviations respectively, dated and explained; all but the last were decided before the measured runs they concern, and two of them reduced E2 on the real-API case to two arms of one sequence each. The last, recorded after a review, is a measure that `protocol-v04.md` required and that was run only after the results (Section 7.2). The protocols number their hypotheses H1–H11; to keep them apart from those of Studies 1 and 2, this paper calls them P4-H1 to P4-H11. As before, a difference inside the other arm's range is not read as a difference, and no significance test is claimed. Table 6 gives every outcome as frozen, those on the strategy included.

Costs. The *LLM API cost per run* is computed from the usage that the model's API billed (input, cached input and output tokens of every turn, so the turns of recovery and the replies of `expand` included) at the model's list prices, and is reported only as a change relative to arm R of the same case. Calls to the tool provider, which the search-data API bills per call, are not in it.

7.1 E1: fixed trimming against raw outputs

Four arms: R (neither strategy text nor trimming), S (strategy text), T (trimming with `expand`) and ST (both), five runs each, on the editorial plan, the content-gap analysis and billing support, with the prompts of Studies 1 and 2; 50 runs in all. Trimming is a fixed configuration learned from the 8 Phase A runs of the case with the library's default rules; on the content-gap case these rules trim no tool, so it has arms R and S only. E1 was designed for the strategy text and states no hypothesis on trimming; arm T, against R, is a live measure of the fixed configuration. This section reads T against R; S and ST are read in Appendix C. Tables 7 and 8 give every arm, and every run is in every figure, including the run that asked a question instead of doing the task.

	Exp.	Hypothesis, as frozen	Outcome	Detail
P4-H1	E1	With the strategy, gates and coverage are no worse	failed	4 of 5 comparisons hold; editorial plan: ST passed the gates in 4/5 runs, T in 5/5
P4-H2	E1	Billing support: median turns with the strategy below the minimum without	failed	median 4 against minimum 3
P4-H3	E1	Removed before freezing (it concerned arms that are not run)	–	–
P4-H4	E1	The strategy does not undo the saving of trimming (billed input)	failed	editorial plan holds; billing support fails
P4-H5	E2	A field recovered with `expand` and written is kept from the next run	holds	16 of 16 recoveries, 10 sequences
P4-H6	E2	A reworded field needs promotion to come back	not testable	reworded in 2 sequences, all with promotion on
P4-H7	E2	Sampling reaches a field the agent does not ask for	failed	runs with a field removed and no `expand`: 1 with sampling, 1 without; requirement met in both
P4-H8	E3	Median turns with the strategy below the minimum without	failed	holds in 2 of 4 cases
P4-H9	E3	Median share of turns with several calls above the maximum without	failed	holds in 0 of 4 cases
P4-H10	E3	Gates and coverage are no worse with the strategy	holds	gates, billing support: 8/8 with, 8/8 without; gates, CI triage: 6/6 with, 6/6 without
P4-H11	E3	The strategy does not override the values the prompts prescribe	holds	9 of 9 checks

Table 6: Study 3: the hypotheses of `protocol-v04.md` (P4-H1 to P4-H7, on E1 and E2) and `protocol-v04-e3.md` (P4-H8 to P4-H11, on E3), stated before the runs, with their outcomes as frozen. The protocols call them H1–H11; the prefix keeps them apart from the hypotheses of Studies 1 and 2. A hypothesis whose precondition did not occur is *not testable*. ST and T: arms of E1 with the strategy and trimming, and with trimming alone. The requirement of P4-H7 is the partial test described in Section 7.3.

Tool-output tokens and cost. Before recovery, the projected tool outputs contained 44% fewer tokens than the same outputs unfiltered in the editorial plan, and 44% fewer in billing support (median of the per-run reductions). Counting also what `expand` sent back, the median reduction was 34% in the editorial plan, where the agent called `expand` in five runs of five (median two calls), and stayed 44% in billing support, where no run called it. The median LLM API cost per run was 30% and 24% lower than in R. That figure comes from billed usage, so it already includes the recovery turns and the `expand` replies; it excludes the calls to the tool provider. The token reductions and the cost change have different denominators (the tool outputs, the whole bill of the model) and are not to be compared or added.

Quality. The gates passed in 5/5 and 5/5 runs of T, as many as in R or more (Table 7). The gates check identifiers and the volumes or facts that the answer attributes to tools, not every association. On billing support the protocol also required a stricter audit, which follows.

7.2 Post hoc: the billing audit that the protocol required

`protocol-v04.md` required, on billing support, the stricter audit of Studies 1 and 2 (Table 12) beside the keyword test. It was not run with the results. We report it now as a deviation from the protocol: run after the results were known, with the rules of `experiments/c4_audit.py` unchanged (`experiments/audit_v04.py`, `results/v04/audits.json`). Like the earlier audit, it applies rules to the diagnosis of each ticket; it is not a reading.

Over the 20 runs of the four arms, 100 diagnoses in all, the audit passed 84; three of its failures are the rule's own (a correct reason next to a negation, which the rule rejects), so that, read one

Case	Arm	Runs	Gates	Turns	Tool calls	`expand` calls	Turns with several calls
Editorial plan	R	5	4/5	9 [8–10]	22 [19–24]	0 [0–0]	75% [67–88]
	S	5	5/5	10 [6–11]	20 [18–26]	0 [0–0]	80% [56–100]
	T	5	5/5	9 [8–11]	24 [17–26]	2 [1–3]	88% [60–100]
	ST	5	4/5	9 [1–12]	23 [0–27]	0 [0–3]	81% [64–89]
Content gaps	R	5	3/5	5 [3–6]	16 [14–20]	0 [0–0]	67% [60–100]
	S	5	4/5	4 [3–6]	16 [13–18]	0 [0–0]	100% [50–100]
Billing support†	R	5	5/5	4 [3–4]	11 [11–13]	0 [0–0]	67% [67–100]
	S	5	5/5	4 [3–4]	12 [10–13]	0 [0–0]	67% [67–100]
	T	5	5/5	3 [3–4]	10 [10–11]	0 [0–0]	100% [67–100]
	ST	5	5/5	4 [3–4]	11 [10–12]	0 [0–0]	67% [67–100]

Table 7: E1, quality and process: `gpt-5.6-terra`, five runs per arm, every run included. Gates: runs in which every gate passed. Turns, tool calls and `expand` calls: median [min–max] per run. *Tool calls exclude* ***`expand`***, counted apart. Turns with several calls: share of the turns with a call that made more than one, per run, median [min–max]; here every call counts, `expand` included. †Simulated tools.

			Tool-output tokens		
Case	Arm	Billed input	before recovery	with `expand` replies	LLM API cost per run
Editorial plan	R	704,930	–	–	–
	S	763,969	–	–	+2%
	T	476,266	−44%	−34%	−30%
	ST	442,252	−44%[a]	−44%[a]	−35%
Content gaps	R	236,839	–	–	–
	S	191,175	–	–	−7%
Billing support†	R	43,275	–	–	–
	S	45,359	–	–	−1%
	T	17,340	−44%	−44%	−24%
	ST	33,613	−43%	−43%	−20%

Table 8: E1, tokens and cost. Billed input: median tokens of a run, as billed. Tool-output tokens: change per run against the same outputs unfiltered, median of the runs; *before recovery* counts what the tools' results became, *with* ***`expand`*** *replies* adds what `expand` sent back. LLM API cost per run: median, relative to R, from billed usage at the model's list prices (recovery included, tool-provider calls excluded). [a]Median of the four runs that called a tool: the run of that arm that asked a question made no call and has no ratio (it is in every other figure). †Simulated tools.

by one, 87 diagnoses are right. The keyword test passed 92, and the gates passed in every run (Table 9). Nearly all the rest is one ticket, and a choice of path: in 11 of the 20 runs, in every arm (one in R, three in S, five in T, two in ST), the model listed that customer's invoices rather than its charges, which hold the reason for the declined payment, and so wrote that the card had failed without saying why (the keyword test accepted five of these). Since this happened without trimming too, it is the model's route, not a field removed; with five runs per arm the data cannot tell whether trimming made that route more likely. The period-end ticket failed in two runs, both with the strategy text.

7.3 E2: adaptive learning

Each case was given a variant of its task that needs two fields its projection removes: in billing support, the link to an invoice and how it is collected; in the editorial plan, a video's channel and its views. A sequence is five consecutive runs in one work directory, seeded with the 8 Phase A runs of the case, with automatic mode on and the base level of the skill. Billing support ran on a copy of its simulated world in which those two invoice fields vary, with promotion off or on

Arm	Runs	Gates	Keyword test	Audit	Audit per run	Decline reason missing
R	5	5/5	25/25	23/25	5 [4–5]	1/5
S	5	5/5	21/25	20/25	4 [3–5]	3/5
T	5	5/5	23/25	20/25	4 [4–4]	5/5
ST	5	5/5	23/25	21/25	4 [3–5]	2/5

Table 9: E1, billing support, post hoc (a deviation: required by `protocol-v04.md`, run after the results). Five tickets per run. Keyword test: the frozen test of the protocol. Audit: the stricter rules of Studies 1 and 2 on the diagnosis alone (Table 12), unchanged; total, and per run as median [min–max]. Decline reason missing: runs whose answer does not say why the payment was declined, read one by one, with the rule's own failures set aside. Readings one by one were made by the AI agent that ran the audit, under the author's review.

and sampling off or on, two sequences per arm (40 runs); the editorial plan ran the arms without promotion, base and sampling, one sequence each (10 runs).

Fields came back by themselves (P4-H5 holds). Each time the model recovered a needed field and wrote its value, the field was in the trimmed outputs from the next run on, with no `expand`: 16 of 16 recovered and written fields were kept in every later run, in ten sequences; from the second run in 8 sequences and from the third in two.

What the requirement test checks. The protocol's test of the added requirement is partial.[1] In the editorial plan it checks that the answer names a channel that the run's YouTube outputs contain; it checks neither the video, nor the views, nor that they belong together. In billing support it checks that a ticket citing an invoice contains a link found in the run's invoice outputs and a keyword about collection; it checks neither that the link is the cited invoice's nor that the method stated is that invoice's. In this sense, and only in it, the requirement was met in 50 of 50 runs.

Post hoc: the full associations. After the results, stricter rules written for the purpose (`experiments/audit_v04.py`) checked the associations on the recorded answers, and confirmed them in nearly all cases. In the editorial plan, for each plan item and each channel of the outputs it names, the item must identify a video of that channel (by title, shortened title or identifier) and state its views as the output gives them, within the precision written: 43 of 46 pairs passed, and every pair was right in six runs of ten. Of the three failures, one is a real misattribution (a video of another channel), one names the right channel and views but writes a title that is not the one the outputs gave, and one is caused by the rule, which takes a channel whose name is a common word as named in the item. In one run the answer names its videos without the words the rule requires ("YouTube", "canale", "channel"), so its three items are not counted; without that condition they pass. In billing support, each ticket citing an invoice must link that invoice's hosted page and state exactly its collection method: 69 of 72 tickets passed the rule. The three failures are the rule's: a reply saying that the invoice is charged automatically and *not* sent for payment is read as stating both methods; read one by one, all 72 are right. These are rules on words, written after the results; a link given in the right reply for a wrong reason would pass them.

Promotion and sampling made no difference here. Promotion exists for a field that is used without being written; this model wrote the values it recovered, so ordinary learning was enough. The field expected to be reworded was reworded in two sequences, both with promotion

[1] The function `requirement()` of `experiments/coverage.py`.

on, so P4-H6 could not be tested. Sampling only cost tokens: in the last run of each billing-support sequence, the median saving on tool-output tokens was 35% with sampling and 42% without. The model never needed it, and P4-H7 failed: in the runs where a needed field was removed and the model did not call `expand` (one with sampling, one without), the requirement, as tested, was met in both, so not more often with sampling.

Another route to a removed field. In those two billing-support runs the model did not call `expand`: it called another tool, which retrieves a single invoice and is not trimmed, and read the fields there. That is a limit of the design on that case (it left P4-H7 little to test), and a behaviour to know: on an API billed per call, that route has a price. In the editorial plan no other tool returns the two fields, and the model used `expand` in both sequences.

One switch-off. In the first run of the editorial plan's base sequence the model expanded two of the four trimmed results of the trends tool, all in one run, and under the rule then in force the tool stopped being trimmed. The saving of that sequence was 33% in its first run and between 26% and 38% later; in the sampling sequence, where no tool was switched off, it stayed between 35% and 40%. An offline recalculation on the ten sequences, not in the protocol, replayed the rule with expansions required from two or three distinct runs: no tool would have been switched off at that point (the recalculation is exact up to the first run in which two rules decide differently; what the agent would have asked afterwards is not in the traces). The library now requires expansions from two distinct runs by default.

Requests that gave nothing back. Of the 38 `expand` requests of E1 and E2, 10 returned nothing. 8 named a result that did not exist (an identifier made up or cut short), 7 of them asking for more rows than the tool had returned: the agent wanted more rows, not hidden fields, which `expand` cannot give. The other two wrote a list index where paths collapse indices; the library now accepts such a path, and answers an unknown identifier by saying how to copy a valid one.

7.4 Offline analyses

Four analyses on runs already recorded, with no new run. The first, the comparison with baselines, was written down before any of its numbers was computed; the other three were in no protocol. All are reported as such.

Does learning add value beyond minification and a static filter? On the Phase A traces of Studies 1 and 2, each run held out in turn, the same calls were passed through five arms (`docs/baselines.md`, `experiments/baselines.py`): *raw*; *minified*, the same JSON written compactly with nothing removed; *static*, a static filter followed by minification; *learned*, the projection as the library now serves it by default (prudent, with the safeguards of Study 2, learned from the other runs of the case); and *both*, the static filter applied after the learned projection. The static filter reads only the payload, never a trace: it removes, at any depth, 35 generic metadata keys (transport envelope, request tracing and caching, type tags and environment flags, pagination, hypermedia links), then null and empty values. Its rules, the arms and the reading rule were committed on their own before any number was computed, as the repository's history shows, and were not changed after. A miss is counted as in Study 1, on the held-out structured calls, the same denominator for every arm; since the learned arm here is the default projection with safeguards, its misses differ from those of Table 2, which counts the aggressive projection of version 0.1. Table 10 gives every case.

Learning adds a large share where payloads are large and the task uses little of them: beyond what the static filter leaves, the learned projection removed 50% in the editorial plan, 40% in the literature scan and 56% in CI triage, with one, two and three misses against none for the static filter. In billing support it added less, 24%, with nine misses. In the content-gap analysis

			Tokens removed, against raw				Learned	Misses		
Case	Calls	Raw tokens	Minified	Static	Learned	Both	vs static	Static	Learned	Both
Editorial plan	167	817,865	−25%	−26%	−63%	−63%	−50%	0	1	1
Content gaps	90	527,542	−9%	−9%	−13%	−13%	−4%	0	8	8
Literature scan	61	2,262,885	−16%	−20%	−52%	−55%	−40%	0	2	2
Billing support†	84	154,833	−33%	−47%	−59%	−69%	−24%	45*	9	48*
CI triage†	64	599,808	−18%	−20%	−65%	−66%	−56%	0	3	3

Table 10: Baselines, offline, on the Phase A traces, each run held out. Calls: held-out structured calls, the denominator of the misses; raw tokens: every call of the held-out runs. Tokens removed against raw: *minified* (compact JSON, nothing removed), *static* (the pre-registered filter, then minified), *learned* (the library's default projection), *both* (static after learned). Learned vs static: what the learned arm removes of the static arm's tokens. Misses: held-out structured calls in which the arm removes a field the run used later; none for raw and minified. *Every static miss is Stripe's type tag, a coincidence of detection (see text). †Simulated tools.

it added little, 4%, with eight misses: there, as Study 2 found, minification is the honest baseline. Minification alone removed 9% to 33% of the raw tokens, a large share of every saving in Table 2 that any serialiser would obtain. The static filter removed little beyond minification where the provider already trims its envelope (2% and 0% of the minified tokens in the two search-data cases) and more where payloads carry nulls, empty lists and type tags (5% in the literature scan, 20% in billing support).

Static and learned are complementary, not rivals. The prudent projection keeps every field whose values the detector cannot judge, nulls and flags among them, which is what the static filter removes: applied after the projection, it added 3.5 points of the minified tokens in the literature scan and 14.9 in billing support. The static filter's misses in billing support are an artefact of detection, not uses: all are Stripe's type tag, `object`, whose value `customer` (40 calls) or `charge` (five) coincides with the name of an argument of the agent's later calls (charges listed with a `customer` argument), which the word matcher reads as reuse of the value. Without that field the static filter has no miss there, and the learned arm keeps the tag for the same reason. This comparison is offline, on the traces on which Study 1 was measured: it counts tokens and detected misses, and says nothing about the quality of live answers, since neither the minifier nor the static filter was run in a live loop.

Projections per mode of use. Would one projection per value of a low-cardinality argument (a result type, a time range, a sort order) do better than one projection per tool? On the 102 recorded runs in which the model saw whole outputs, each held out in turn, 18 candidate arguments on 11 tools were evaluated, identifiers left out. The rule, written before the numbers: a benefit is at least five more points of the tool's tokens removed without more calls losing a field the run used. A benefit was found for no candidate. The most any candidate added was 3.2 points, and 16 added less than one; five lost a used field in more calls than the projection per tool, because a profile learns from fewer calls; 8 had a profile with fewer than three calls, and 35 held-out calls fell on a profile too thin to have its own projection. The specialisation was not implemented.

Rows. The library removes fields, never rows. In the 112 recorded runs in which the model saw whole results, on the search-data API the elements of a list that the run reused afterwards were exactly the first ones in 25% of the calls with any reuse, and keeping only the first five would have removed a reused element in 92% of them; on the other APIs, whose lists are short, the figures are 98% and 2%. On the search-data API a cut by position would remove elements the agent reuses, and a removed row, unlike a removed field, cannot be told from one that does not exist. No cut of rows is implemented; the telemetry report shows how many list elements are reused, and from where.

Tool definitions. The model receives the definition of every listed tool at every turn. Counted on the schemas as sent in the 138 recorded runs of E1, E2 and E3, in the reference arms the definitions were 1.1% to 10.9% of the input tokens (median per case and model), but, sitting in the cached head of the prompt, 0.2% to 8.5% of the LLM API cost of a run at the providers' list prices; had nothing been cached, up to 31.5%. A counterfactual, with no run made, asks what a whole server would weigh: the search-data provider's MCP server lists 79 tools, of which the editorial-plan agent was given 12 and called 11. Taking its five recorded R runs as they happened and changing only the definitions, listing every tool would send 41,944 tokens per turn instead of 7,699; listing the tools called and putting the other 68 behind a search tool and a call tool, with no search ever made (the most hiding could gain), 7,808, 81% fewer than the whole server, and the first turn would go from 44,779 to 10,643 tokens. With a cache, depending on what is assumed cached, the LLM API cost of a run would fall by 9% to 20%, and 1.9 to 5.2 extra turns (medians) would cancel the gain; the scenario also assumes that the model works with the whole server as it did with its own tools. The library therefore measures the definitions and does not hide tools: its report lists the tools never called, with their weight, and suggests disabling them in the host or using native search where it exists. Some hosts already defer definitions: Claude Code loads MCP tools on demand by default [1]; the OpenAI Responses API offers tool search when the request asks for it [16]; Cursor does so according to a blog post, which we did not verify in its documentation [3]. A second search on top of theirs would add turns, and a hidden tool called through a generic call tool would bypass the per-tool permissions that hosts grant.

7.5 The strategy text, and the hosts

The strategy text was measured in E1, in arms S and ST on `gpt-5.6-terra`, and in E3, alone and without trimming, on `gemini-3.8-flash`; the proxy was also checked in several MCP hosts. Appendix C reports both in full, with Table 15 and a post-hoc analysis. In short: on the first model the strategy alone (S against R) stayed inside R's range in 12 of 12 comparisons, while combined with trimming (ST against T) the results were mixed, and P4-H1, P4-H2 and P4-H4 failed; on the second model it shortened the runs of CI triage (median turns from 17 to 6), and P4-H8 and P4-H9 failed as stated.

8 Discussion

What the current version delivers. With the defaults the library ships today (identifiers protected, transformed values recognised, compact JSON, `expand`), trimming paid off in live runs: the LLM API cost per run fell by 30% and 24% on the two agents measured, recovery included, with every implemented gate passing. Learning kept up with a changed task by itself: fields the agent recovered and wrote were kept from the next run in 16 of 16 cases, with no correction needed. And what the learned projection removes goes well beyond the tested static baseline: beyond a minifier and a static metadata filter it removed 40% to 56% of what was left on three of five agents.

What the agent uses can be learned, best from complete traces. On four of the five agents, most of what tools return is never visibly used, and a projection learned from a handful of runs of the same task removes most of it on a new run. In these agents the learner needed the final answer: an MCP gateway placed between agent and tools can *apply* a projection, but learning from what it sees alone misses far more of what the agent uses (with the aggressive projection of version 0.1, 49% of the held-out structured calls it trims against 4%, on the three agents with real APIs). The practical consequence is an architecture: learn offline from full traces, review, apply anywhere. Version 0.3 of the library adds a second route, not evaluated in Studies 1 and 2: the proxy offers the agent one more tool, through which the agent hands over its final answer, so that the gateway sees complete traces and can learn and apply the safe part of a projection on

its own. Study 3 measured automatic learning through the function-calling adapter, not through the proxy; in the hosts checked, the agent did call that tool.

Where usage detection is blind. The losses we found have one cause: the projection removes what the detector cannot see. The agent transforms values (timestamps into dates, cents into euros, URLs into short identifiers, English into Italian), or reads a field only to decide something (an issue title to match a failure). Safeguards can close known blind spots, and S1 and S2 closed three of the four we found, but the list of possible transformations is open. The general lesson is that a learned projection is a *proposal*, whose blind spots are exactly the fields that matter in ways the text does not show.

Identifiers are special. The most serious failure was not a loss but a fabrication: deprived of an identifier it was asked to report, an agent invented plausible ones. Identifiers are cheap to keep and costly to lose, so a projection should never remove them, whatever the usage says.

The recovery path matters. With `expand`, a wrong cut can cost one round trip instead of a wrong answer: in the live runs the agents asked for hidden fields, and the LLM API cost per run, which includes the recovery turns, still fell. It is not a guarantee: in one billing-support run the agent did not ask for the period end and got a ticket wrong. It works only when the agent knows what is hidden, which is why the hidden fields are listed, grouped, after every trimmed output, and only while the original is kept: the library keeps the last 256 originals per proxy and lists at most 25 groups, summarising the rest.

How to use it. These results suggest a workflow rather than an automatic switch: record a few runs of a task with full outputs; generate the telemetry report, which is useful on its own and changes nothing; review the proposed configuration, a plain list of kept and dropped fields; apply it with identifier protection and `expand`; and keep recording, so that a field the agent starts needing shows up in the next report. The library implements each step, including an MCP proxy that applies a configuration in front of any server without changes to the agent. Reported savings should also be read against re-serialisation alone, which removes up to 13% of the real-API outputs and 17% of the simulated ones without removing anything, and against a full minifier.

What the baselines add. The offline comparison of Section 7.4 separates three things that the savings of Studies 1 and 2 mix: what any serialiser removes (minification, 9% to 33% of the raw tokens), what a filter of generic metadata and empty values removes without looking at any trace, and what the learned projection removes beyond that tested static baseline. Where payloads are large and the task uses little of them, learning removed most of what was left after the static filter (40% to 56% in three cases); where the agent cites most of what it reads, as in the content-gap analysis, it added little, and minification is the honest choice. The static filter is a complement, not a rival: it removes the nulls and flags that the prudent projection keeps because they can never be judged. The comparison is offline: it shows where learning can add value in tokens, not that the answers stay as good, which only live runs can show.

What else Study 3 shows. Counting what `expand` sent back, the tool outputs shrank less where the agent recovered fields (34% against 44% before recovery, in the editorial plan), and the cost figure already includes that. The two corrections designed for harder cases of adaptation, promotion and sampling, were not needed in these runs, and sampling only cost tokens, so both stay off by default. The agent also found a route around a removed field that no part of the library sees, another tool, which on an API billed per call has a price. The stricter post-hoc checks confirmed the answers in nearly all cases; the one recurring gap, a decline reason missing in billing support, came from the model's choice of which records to list and occurred in every

arm, with or without trimming (Section 7.2). The strategy text (in the appendix) is useful for models that issue calls one at a time: it shortened the runs of one agent of such a model, and changed nothing measurable against R for a model that already grouped its calls (combined with trimming, its results there were mixed), so it stays off by default and the telemetry report suggests it when the traces show calls made one per turn that did not have to wait. The negative results are as informative. Measured offline on these runs, specialising projections by argument value gained nothing, cutting rows by position would have removed elements the agent reuses, and hiding unused tool definitions would gain a modest share of the cost where they are cached, which a few extra turns would cancel.

9 Limitations

- **Validated on the development tasks only.** The tasks, prompts, gates, safeguards and default rules were developed on the same five agents on which they are evaluated, and Studies 2 and 3 reuse them. No result has yet been tested on tasks, agents or APIs kept apart from development: the results describe these workloads, and validation on new ones is the main step still missing.
- **What the checks check.** The gates, the protocols' tests and the rule-based audits added afterwards check identifiers, links, names, keywords and some relations (a volume against its keyword, a link against the invoice cited, a view count against its video), not every association an answer makes. The stricter post-hoc audits confirmed nearly all answers, and their exceptions, such as a decline reason the agent never fetched, passed the gates (Section 7.2).
- **No independent human rating.** Answers were checked by gates and rules; where a check failed, a rule seemed wrong or a result was unexpected, they were read one by one by the AI agents that ran the analyses, under the author's review. No independent human rater read every answer, and no agreement between raters was measured.
- **Small numbers of runs.** Eight live runs per case to learn, two to five replays per arm to judge quality. No significance test is claimed; differences inside the range of the raw arm are reported as not distinguishable from run-to-run variation, which in these tasks is large (in Phase C the raw arm itself misdiagnosed a ticket in two of three runs).
- **One model.** All runs of Studies 1 and 2 used `gpt-5.6-terra` at default parameters. Whether other models use tool outputs differently, or read projected and TOON-encoded outputs differently, is untested.
- **Replay is not a live loop.** Phases B and Study 2 replay a fixed sequence of calls up to the final answer; in a live loop a projected output can also change which calls come next. Phase C measured the live loop on two cases only.
- **Simulated cases.** Two of the five cases use payloads we generated from public OpenAPI descriptions. Their shape is realistic, their content is not, and because we designed them they are reported separately.
- **Usage is detected reuse.** Reasoning is not observable in the APIs used, so fields used only while reasoning are invisible (the root of the failures reported, and safeguards cover only the blind spots that are known); coincident values can also make a field look used. The detector's precision and recall were not measured against an annotated sample.
- **Study 2 is not an independent validation.** The safeguards were designed on the Phase B failures and tested on the same conversations; new tasks, runs and seeds would be needed to show that they generalise.

- **Keyword tests.** The thermometers of billing support and CI triage test for keywords and numbers; they accept a negated or contradictory sentence. A stricter audit of the billing diagnoses is reported beside them.
- **Post hoc decisions.** Three measurement corrections were made after seeing data (deviations 8, 10 and 11); each is applied to every run, and the strict test of deviation 11 is reported beside the frozen one. The gates' reading of projected outputs and the re-check of the Phase C volumes (deviation 13) were also fixed or added afterwards.
- **What the gates cannot see.** The editorial-plan gate checks a volume only when the run's outputs give one for that keyword, so a volume written without a source passes it; a post-hoc re-check found one in the live runs (Section 5.5).
- **Lossless part of the savings.** Every saving reported includes re-serialisation (compact JSON without escapes), which removes nothing; it is reported separately so that the projection is credited only with the rest.
- **Number formats.** Usage detection in the studies matched numbers written plainly or with dot thousands separators; numbers written with comma separators would have gone unseen. In all Phase A runs the agents wrote 0 such numbers, so no use was missed this way; the library matches them since version 0.3.4.
- **Study 3.** Trimming and adaptive learning were measured on one model, `gpt-5.6-terra`, the strategy text on two; other models may organise calls, call `expand` or reword values differently. Runs are few: five per arm in E1; in E2 two sequences per arm on the simulated case and one on the real-API case, a single trajectory per arm; two to eight per arm in E3, with one task per case and no trimming, so the strategy combined with trimming was not measured on the second model. In E3 the editorial plan and the content-gap analysis ran on recorded outputs, mostly the nearest recorded call rather than the same one, so only the organisation of calls is evaluated there. The gates and the test of E2's added requirement look for identifiers, links, names and keywords; the full associations and the billing audit were checked only after the results, by rules, with the cases those rules got wrong read one by one. The audit that `protocol-v04.md` required for billing support was run after the results, a deviation. Only the function-calling adapter was exercised. The offline analyses, the baselines among them, use the traces on which the method was developed and say nothing about live answers; apart from the baselines, whose rules were fixed before any number, they and the post-hoc analysis of E3 were not in a protocol, and the host checks are few observations per host. Every attempt is kept in every figure, the run that asked a question instead of doing the task included; a ratio that such a run cannot have is computed without it and says so.
- **Third-party data.** Search results and parsed pages cannot be redistributed: for the two cases built on them, the released data are derived per-call records: enough to recompute which fields each call used and the misses, but neither the token savings, which need the payloads, nor a replay of the conversations.

10 Conclusion

With the defaults it ships today, a projection learned from an agent's own traces makes agents cheaper to run: in live runs on one model, a fixed projection with `expand` lowered the median LLM API cost per run by 30% and 24% on two agents. All implemented gates passed, but the stricter billing audit found less complete diagnoses in the trimmed arm (20/25 against 23/25 without trimming); preservation of answer quality has not been established. Fields that a changed task needed came back by themselves from the next run in 16 of 16 cases, and, offline, learning

removed 40% to 56% of what a minifier and a static filter leave on three of five agents. The studies with earlier versions show where these defaults come from.

Agents read far more than they visibly reuse, and what they reuse in writing can be learned from their own traces: with today's defaults the learned projection removes 63%, 52%, 59% and 65% of held-out tool-output tokens on four of the five agents (offline, Table 10); in Study 1, on the same four, the prudent projection of version 0.1 removed 63% and 70% of the tool-output tokens of a new run on two agents with real APIs, and 69% and 77% on two simulated ones (re-serialisation alone, which removes no field, accounting for up to 13% and 17%). Where it failed, it failed for a reason: the agent had transformed a value, used a field only to reason, or, deprived of an identifier, invented one. The two safeguards of version 0.2, keeping identifying fields and recognising transformed values, repaired the specific failures that motivated them, in the conversations where they were found, while keeping a saving of 53% and 46% on those real-API agents and 57% and 61% on the simulated ones; measured against outputs already minified, it still removed 39% and 35% of the tokens on those real-API agents and 36% and 53% on the simulated ones, while on the agent that cites most of what it receives a minifier alone did better; a fourth mechanism, translation, remains open. In short: a projection learned from textual reuse can shrink already minified outputs in several workloads; the quality of the answers depends on information that this reuse makes only partly observable, and the safeguards correct some of the documented failures. The method is therefore best used as it is implemented: as a proposal learned from complete traces, reviewed by a person, applied with identifiers protected and with a way back for the agent when a cut was wrong, which in Study 3 also corrected the projection. It helps most where tool outputs carry much that the task does not use, and least where the agent cites most of what it reads, where minifying is enough. The evidence comes from few runs, one model for trimming and learning, and the tasks on which the method was developed; validation on new tasks is the next step.

Version history

Version 1 (6 November 2026), subtitled *and Where They Fail*: Studies 1 and 2. Version 2 (November 2026), subtitled *and When They Pay Off*, since it measures what the method saves in live runs and against simpler baselines: adds Study 3 (Section 7), on the measures made after version 1, with its post-hoc audits, and its place in the introduction, discussion, limitations and conclusion; adds a comparison with related work (Table 1), the definitions of detected reuse, required information and projection miss (Section 3.1), an offline comparison with a minifier and a pre-registered static filter (Section 7.4) and a summary of what can be checked independently (Table 11). In Studies 1 and 2 no number, result or conclusion changed; version 2 clarifies some captions and sentences (the abbreviations of the audit table, the version, configuration and denominator of the misses, the scope of the safeguards' repair and of the comparison with Headroom), and sentences elsewhere that referred to "the studies" now name Studies 1 and 2. The abstract was shortened.

AI assistance and authorship

The author set the research question, the constraints of the study (own domains only, providers whose terms allow publication) and the requirement that the result be a usable tool, and approved each protocol before it was frozen. An AI coding agent (Claude Code, model Claude Opus), working in the author's project under the author's direction, proposed the protocols, wrote the library and the experiment code, ran the experiments and the analyses, and drafted this manuscript; a sub-agent compiled the bibliography, each entry verified against a primary source. The simulated payloads are generated by code from public OpenAPI descriptions, not written by a model. No human annotation was performed beyond the author's review. The author is responsible for the

decision to publish and for the final text. The work added in version 2, Study 3 and the software from version 0.4 on, was carried out in the same way, with AI coding agents (Claude Code) working under the author's direction, the author deciding its scope, approving its protocols and deciding every publication.

Funding and competing interests

This study received no external funding. API costs were paid by the author. The author has no financial or other relationship with OpenAI, Google, Anthropic, Microsoft, DataForSEO, OpenAlex, Stripe, GitHub or the authors of Headroom and TOON.

Code and data availability

The library (`tracetrim`, MIT licence), the experiment code, the frozen protocols with all deviations, the aggregate results and the scripts that generate every number and table of this paper are released at `https://github.com/alepom/tracetrim` and archived on Zenodo with the dataset (DOI 10.5281/zenodo.24190053). Traces of the literature-scan case (CC0 catalogue) and of the two simulated cases are released in full. For the two cases built on the search-data API, whose terms do not license redistribution of results, only derived per-call records are released: tool, field paths used, echoed and judgeable, every field path seen, and token counts of the raw outputs. They are enough to recompute which fields were used and the misses without the provider's data (`experiments/misses_from_derived.py` does, and checks the result against Table 2), but not the token savings of those two cases, which need the payloads. The simulated servers and the payload generator are included, with the exact versions of the OpenAPI descriptions they follow. For Study 3 the repository adds the two frozen protocols with their deviations (`protocol-v04.md`, `protocol-v04-e3.md`), the aggregate results (`results/v04`, `results/v06`) and a summary of the measures (`docs/measures-v04.md`); the traces of the two cases built on the search-data API, and the recorded outputs of E3, are not redistributable either. Table 11 summarises what an independent reader can check, and with what.

Status	Results
Replicable from open material	Studies 1 and 2 on the literature scan, billing support and CI triage (full traces, simulated servers, replay code); Study 3 on billing support and CI triage, with the traces of the dataset; the offline baselines on the same three cases.
Recomputable only in part, from derived records	The editorial plan and the content-gap analysis: which fields each call used, the misses and the token counts of the raw outputs, from the per-call records; not the token savings, the replays or the baselines' token figures, which need the payloads.
Not reproducible publicly without access to the provider	The live outputs of the search-data API in the editorial plan and the content-gap analysis, in every study, and the recorded outputs of E3: rerunning those cases needs an account with the provider and gives other outputs.

Table 11: What can be checked independently, and with what. The library, the experiment code, the protocols and the aggregate results are open in every case.

References

[1] Anthropic. Claude code documentation, page on MCP. `https://code.claude.com/docs/en/mcp`, 2026. Sections "Scale with MCP tool search" and "Configure tool search". Accessed

2026-11-12.

[2] Atlassian Labs. mcp-compressor: An MCP server wrapper for reducing tokens consumed by MCP tools. https://github.com/atlassian-labs/mcp-compressor, 2026. Version 0.40.0 (latest release at access date). Apache-2.0. Accessed 2026-11-04.

[3] Cursor. Dynamic context discovery. https://cursor.com/blog/dynamic-context-discovery, 2026. Blog post, not the product documentation. Accessed 2026-11-12.

[4] Headroom Contributors. Headroom: Compress tool outputs, logs, files, and RAG chunks before they reach the LLM. https://github.com/headroomlabs-ai/headroom, 2026. Python package headroom-ai, Apache-2.0. The version used for the measurements is stated in the text. Accessed 2026-11-04.

[5] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.825. URL https://aclanthology.org/2023.emnlp-main.825/.

[6] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.91. URL https://aclanthology.org/2024.acl-long.91/.

[7] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55321–55375, 2026. URL https://proceedings.mlr.press/v306/kang26b.html.

[8] Ádám Kovács. Squeez: Task-conditioned tool-output pruning for coding agents, 2026. URL https://arxiv.org/abs/2604.04979.

[9] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.391. URL https://aclanthology.org/2023.emnlp-main.391/.

[10] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management, 2025. URL https://arxiv.org/abs/2508.21433. Presented at the 4th Deep Learning for Code (DL4C) Workshop at NeurIPS 2025.

[11] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

[12] Yuetong Liu, Zhehao Zhou, Yitong Zhang, Haorui Chen, and Jia Li. Tools need not speak in text: Latent context compression for agents. arXiv:2611.06500, 2026.

[13] lourencomaciel. Sift: Reliability gateway for AI tool output. https://github.com/lourencomaciel/sift-gateway, 2026. Python package sift-gateway, version 0.4.4, https://pypi.org/project/sift-gateway/. MIT. Accessed 2026-11-04.

[14] Model Context Protocol. Model context protocol specification, version 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28, 2026. Latest version as of access date; source at https://github.com/modelcontextprotocol/modelcontextprotocol. Accessed 2026-11-04.

[15] Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 19327–19352, 2023. doi: 10.52202/075280-0848. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/3d77c6dcc7f143aa2154e7f4d5e22d68-Abstract-Conference.html.

[16] OpenAI. OpenAI API documentation, guide on tool search. https://developers.openai.com/api/docs/guides/tools-tool-search, 2026. Accessed 2026-11-12.

[17] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.57. URL https://aclanthology.org/2024.findings-acl.57/.

[18] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html.

[19] Ritul Satish, Prasoon Sinha, Akiho Kawada, and Neeraja J. Yadwadkar. Beyond token savings: A systematic study of context compression in LLM agents. arXiv:2609.32961, 2026.

[20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 68539–68551, 2023. doi: 10.52202/075280-2997. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

[21] Johann Schopplich. TOON specification: Token-oriented object notation, version 4.4 (working draft). https://github.com/toon-format/spec, October 2026. Dated 2026-10-08. Reference implementation: https://github.com/toon-format/toon. Accessed 2026-11-04.

[22] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3687–3706. USENIX Association, 2025. ISBN 978-1-939133-52-6. URL https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen.

[23] William H. Walters and Esther Isabelle Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13(1):14045, 2023. doi: 10.1038/s41598-023-41032-5.

[24] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL `https://proceedings.iclr.cc/paper_files/paper/2024/hash/bda88ed2892f5e61c9a9bf215c566913-Abstract-Conference.html`.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL `https://arxiv.org/abs/2210.03629`.

A Deviations from the protocols

All are dated and explained in full in the repository's `protocol.md`; none was hidden or left unapplied.

0. A bug in the agent loop dropped the system prompt after the first turn; found in the pilot and fixed before Phase A. Pilot runs are excluded.
1. Prompts were written during the pilot rather than at freezing, and fixed before Phase A.
2. Tool-level caps made explicit: at most five results per literature search (amendment A2; this deviation had first set ten); a safety guard at 80 calls.
3. The literature catalogue was used without an API key, within its free daily allowance.
4. A verification pilot after the fix; one correction to the CI-triage gate (several job names per cause).
5. Simulated tools fixed before Phase A, with the OpenAPI versions and the presence rule documented.
6. Transport errors of the search-data server: retries reconnect; after three failures the agent receives an error and the run continues, flagged. Two editorial-plan runs of Phase A and two of Phase C (one per arm) are flagged; the results without them are reported in Section 5.
7. The list of hidden fields shown with `expand` was grouped, largest groups first, before Phase C.
8. Post hoc: the literature-scan gate now requires a declared identifier to exist (it had accepted a valid DOI next to an invented identifier).
9. Phase C was delayed by a faulty resume script; no run was altered.
10. Post hoc: the CI-triage gate also splits job names on "/".
11. Post hoc: a strict test for one billing ticket, reported beside the frozen one; a defect in its pattern (it also accepted "fondi sufficienti") was fixed in the publication review, which changes one Phase A run.
12. Traces of the search-data cases are released as derived per-call records instead of pseudonymised traces, which could not have preserved the analysis.
13. Post hoc: the gates did not read projected outputs, which end with the line of hidden fields; the line is now cut and every Phase C run re-checked (no gate result changed). The editorial-plan volumes are also re-checked against the raw outputs of the other runs of the same day (Section 5.5).

14. Post hoc, publication review: usage compared by model turn. Calls emitted in one turn were compared by their position in the trace, so a later call's arguments could count as using an earlier sibling's output. Recomputed by turn, the detected usage changes in 18 of 558 Phase A calls and every Study 1 figure is unchanged; the replay projections are identical.

15. Post hoc, publication review: thermometers and audits. The thermometers were implemented after the protocol was frozen and before Phase A, and differ from its list (C1 lacked the mean volume; its "topics covered" relied on the course's list of expected themes and cannot be computed here). H2 is also evaluated on the computable part of the protocol's list (Tables 3 and 5), with the same outcome in every arm. The gates are narrower than the protocol's wording, and the billing thermometer is a keyword test: both are measured in Appendix B.

B Supplementary results

What the gates do not check. Over all runs, the editorial-plan gate skipped three of 2,535 reported keyword–volume pairs (keywords without a volume in the run's outputs); in the content-gap analysis 11 of the 371 cited results found in the search output carry a title different from the one returned (the gate checks only the URL); in the literature scan no declared DOI differs from the DOI of the work with that identifier. The literature-scan gate in its original form, before deviation 8, passed the prudent, aggressive and structural arms of Phase B in 2/2, 2/2 and 2/2 runs, against 0/2, 0/2 and 1/2 with the corrected gate (`experiments/gates_audit.py`).

Runs	n	Frozen test	Audit
A	8	38/40	33/40
B aggressive	2	8/10	6/10
B prudent	2	8/10	4/10
B raw	2	10/10	10/10
B structural	2	7/10	6/10
C prudent	3	13/15	12/15
C raw	3	13/15	12/15
D raw	5	25/25	23/25
D v02	5	25/25	20/25
D v02 structural	5	22/25	20/25

Table 12: Billing support: tickets passing the frozen keyword test and the stricter rules on the diagnosis alone (exact amounts, year and outcome of the scenario, negations rejected; `experiments/c4_audit.py`, deviation 15). Runs: A, B and C are Phases A, B and C of Study 1, with their arms; D is Study 2, the replays with the safeguards; n is the number of runs, each with five tickets. Rules, not a reading: they show what the keyword test lets through, not that a diagnosis is right. The same audit for Study 3 is in Table 9.

Case	Re-serialised	Minified	0.1 prudent	0.2	0.2 vs re-serialised	0.2 vs minified	Headroom
Editorial plan	−12%	−24%	−59%	−53%	−47%	−39%	−28%
Content gaps	−5%	−10%	−12%	−9%	−4%	+1%	−10%
Literature scan	0%	−16%	−68%	−46%	−46%	−35%	−17%
Billing support†	−17%	−33%	−69%	−57%	−49%	−36%	−33%
CI triage†	−7%	−18%	−77%	−61%	−58%	−53%	−31%

Table 13: Tool-output tokens on the replayed conversations, change against the raw outputs: re-serialised as tracetrim writes JSON (default separators), fully minified, the 0.1 prudent projection, the 0.2 projection; then the 0.2 projection against the re-serialised outputs and against the fully minified ones (what removing fields adds beyond each lossless rewriting); and Headroom, as an illustration of tokens only in the configuration chosen.

Study	Case	Arm	Runs	Thermometers: median [min–max]
B	Editorial plan	raw	2	items 20 [20–20]; median volume 455 [320–590]; mean volume 2711.5 [2357–3066]; long tail pct 5 [3–7]; notes with blog link pct 45 [45–45]
B	Editorial plan	prudent	2	items 22 [20–24]; median volume 590 [590–590]; mean volume 2630 [2431–2829]; long tail pct 3.5 [3–4]; notes with blog link pct 36.5 [33–40]
B	Editorial plan	aggressive	2	items 23 [22–24]; median volume 1300 [1300–1300]; mean volume 3178.5 [3154–3203]; long tail pct 1.5 [1–2]; notes with blog link pct 37 [32–42]
B	Editorial plan	structural	2	items 22 [20–24]; median volume 1300 [1300–1300]; mean volume 2737.5 [2669–2806]; long tail pct 4 [4–4]; notes with blog link pct 44 [38–50]
B	Content gaps	raw	2	topics 3 [3–3]; distinct sources 18.5 [16–21]; own pages read 3 [3–3]; gaps 10 [10–10]; gaps with evidence pct 100 [100–100]
B	Content gaps	prudent	2	topics 3 [3–3]; distinct sources 18 [18–18]; own pages read 3 [3–3]; gaps 9.5 [9–10]; gaps with evidence pct 100 [100–100]
B	Content gaps	aggressive	2	topics 3 [3–3]; distinct sources 18 [18–18]; own pages read 3 [3–3]; gaps 11 [10–12]; gaps with evidence pct 100 [100–100]
B	Content gaps	structural	2	topics 3 [3–3]; distinct sources 16.5 [15–18]; own pages read 3 [3–3]; gaps 10 [10–10]; gaps with evidence pct 100 [100–100]
B	Literature scan	raw	2	works 12 [12–12]; distinct works 12 [12–12]; recent 2023 pct 100 [100–100]; open questions 6 [6–6]
B	Literature scan	prudent	2	works 12 [12–12]; distinct works 12 [12–12]; recent 2023 pct 100 [100–100]; open questions 7 [7–7]
B	Literature scan	aggressive	2	works 11.5 [11–12]; distinct works 11.5 [11–12]; recent 2023 pct 100 [100–100]; open questions 6 [6–6]
B	Literature scan	structural	2	works 11.5 [11–12]; distinct works 6.5 [1–12]; recent 2023 pct 100 [100–100]; open questions 6 [6–6]
B	Billing support†	raw	2	tickets 5 [5–5]; correct diagnoses 5 [5–5]; correct diagnoses strict 5 [5–5]; evidence per ticket 1.8 [1.8–1.8]
B	Billing support†	prudent	2	tickets 5 [5–5]; correct diagnoses 4 [4–4]; correct diagnoses strict 3 [3–3]; evidence per ticket 2.4 [2.4–2.4]
B	Billing support†	aggressive	2	tickets 5 [5–5]; correct diagnoses 4 [4–4]; correct diagnoses strict 3 [3–3]; evidence per ticket 1.8 [1.8–1.8]
B	Billing support†	structural	2	tickets 5 [5–5]; correct diagnoses 3.5 [3–4]; correct diagnoses strict 3 [3–3]; evidence per ticket 1.8 [1.6–2]
B	CI triage†	raw	2	root causes 3 [3–3]; causes found 3 [3–3]; links found 4 [4–4]; failed runs covered 5 [5–5]
B	CI triage†	prudent	2	root causes 3 [3–3]; causes found 3 [3–3]; links found 3 [3–3]; failed runs covered 5 [5–5]
B	CI triage†	aggressive	2	root causes 3 [3–3]; causes found 3 [3–3]; links found 3 [3–3]; failed runs covered 5 [5–5]
B	CI triage†	structural	2	root causes 3 [3–3]; causes found 3 [3–3]; links found 3 [3–3]; failed runs covered 5 [5–5]
2	Editorial plan	raw	5	items 22 [20–24]; median volume 590 [355–590]; mean volume 2813 [2464–2874]; long tail pct 3 [2–5]; notes with blog link pct 40 [32–46]
2	Editorial plan	v02	5	items 20 [20–24]; median volume 1300 [1300–2900]; mean volume 2970 [2736–3345]; long tail pct 3 [1–4]; notes with blog link pct 35 [33–45]

Study	Case	Arm	Runs	Thermometers: median [min–max]
2	Editorial plan	v02 structural	5	items 20 [20–24]; median volume 590 [390–1300]; mean volume 2634 [2401–2967]; long tail pct 3 [2–5]; notes with blog link pct 36 [29–40]
2	Content gaps	raw	5	topics 3 [3–3]; distinct sources 16 [12–18]; own pages read 3 [3–3]; gaps 9 [9–10]; gaps with evidence pct 100 [100–100]
2	Content gaps	v02	5	topics 3 [3–3]; distinct sources 18 [15–18]; own pages read 3 [3–3]; gaps 10 [9–10]; gaps with evidence pct 100 [100–100]
2	Content gaps	v02 structural	5	topics 3 [3–3]; distinct sources 17 [15–21]; own pages read 3 [3–3]; gaps 10 [9–12]; gaps with evidence pct 100 [100–100]
2	Literature scan	raw	5	works 11 [11–12]; distinct works 11 [11–12]; recent 2023 pct 100 [100–100]; open questions 6 [6–7]
2	Literature scan	v02	5	works 12 [11–12]; distinct works 12 [11–12]; recent 2023 pct 100 [100–100]; open questions 6 [6–7]
2	Literature scan	v02 structural	5	works 11 [11–11]; distinct works 11 [11–11]; recent 2023 pct 100 [100–100]; open questions 6 [6–7]
2	Billing support†	raw	5	tickets 5 [5–5]; correct diagnoses 5 [5–5]; correct diagnoses strict 5 [4–5]; evidence per ticket 1.8 [1.8–1.8]
2	Billing support†	v02	5	tickets 5 [5–5]; correct diagnoses 5 [5–5]; correct diagnoses strict 4 [4–4]; evidence per ticket 1.8 [1.6–1.8]
2	Billing support†	v02 structural	5	tickets 5 [5–5]; correct diagnoses 4 [4–5]; correct diagnoses strict 4 [4–4]; evidence per ticket 1.8 [1.6–1.8]
2	CI triage†	raw	5	root causes 3 [3–3]; causes found 3 [3–3]; links found 4 [4–4]; failed runs covered 5 [5–5]
2	CI triage†	v02	5	root causes 3 [3–4]; causes found 3 [3–3]; links found 4 [4–4]; failed runs covered 5 [5–5]
2	CI triage†	v02 structural	5	root causes 3 [3–3]; causes found 3 [3–3]; links found 4 [4–4]; failed runs covered 5 [5–5]
C	Editorial plan	raw	3	items 20 [20–20]; median volume 1600 [260–1600]; mean volume 12550 [1051–21655]; long tail pct 10 [0–10]; notes with blog link pct 40 [35–40]
C	Editorial plan	prudent	3	items 20 [20–20]; median volume 535 [260–1450]; mean volume 1486 [514–465808]; long tail pct 8 [7–20]; notes with blog link pct 40 [30–45]
C	Billing support†	raw	3	tickets 5 [5–5]; correct diagnoses 4 [4–5]; correct diagnoses strict 4 [4–4]; evidence per ticket 1.6 [1.4–1.8]
C	Billing support†	prudent	3	tickets 5 [5–5]; correct diagnoses 5 [3–5]; correct diagnoses strict 4 [3–5]; evidence per ticket 1.6 [1.4–2]

Thermometers per arm, median [min–max]: B is Phase B, 2 is Study 2, C is Phase C.

C The strategy text, and the hosts checked

This appendix holds the part of Study 3 that concerns the strategy level of the skill, a text that teaches the agent how to organise its tool calls, and the checks of the proxy in MCP hosts (Section 7.5 summarises them). The hypotheses are numbered as in Table 6, where every outcome is given.

C.1 E1: the strategy text on the first model

The strategy alone changed nothing measurable on this model. Against the arm without it, the median of S stayed inside R's range for turns, tool calls, billed input tokens and the share of turns with several calls in 12 of 12 comparisons (three cases, four measures; Tables 7

Case	Arm	Runs	Turns	Tool calls	Turns with several calls	Billed input	LLM API cost per run
Billing support†	without	8	3.5 [3–8]	11.5 [10–14]	88% [43–100]	25,316	–
	strategy	8	3 [3–3]	10 [10–12]	100% [100–100]	17,196	−20%
CI triage†	without	6	17 [5–19]	16 [13–18]	0% [0–100]	865,862	–
	strategy	6	6 [5–8]	16.5 [15–20]	90% [57–100]	391,018	−15%
Content gaps‡	without	3	6 [6–7]	15 [14–16]	100% [100–100]	169,493	–
	strategy	3	5 [4–7]	14 [13–19]	100% [100–100]	131,489	+1%
Editorial plan‡	without	2	14 [13–15]	23 [21–25]	47% [36–58]	1,023,210	–
	strategy	2	10.5 [10–11]	18 [17–19]	53% [50–56]	658,204	−2%

Table 15: E3, `gemini-3.8-flash`, without and with the strategy text, no trimming. Turns, tool calls and turns with several calls as in Table 7; billed input as in Table 8; LLM API cost per run relative to the arm without the strategy, from billed usage at the model's list prices. Billed tokens are not comparable with E1's (another tokenizer, price list and caching). †Simulated tools, live. ‡Recorded outputs in place of the provider: calls and turns only.

and 8). This holds for S against R and for these measures only. There was little to organise: without the strategy, 77%, 68% and 77% of the turns that made a call made several (editorial plan, content gaps, billing support; 78%, 76% and 77% with it).

Combined with trimming, the results were mixed, and the hypotheses on the strategy failed as frozen. P4-H1 failed in one comparison of five: in the editorial plan ST passed the gates in 4/5 runs against 5/5 for T (the run that failed is the one that asked a question). P4-H2 failed: on billing support the median turns of S were 4, not below R's minimum of 3. P4-H4 failed on billing support, where the median billed input of ST (33,613 tokens) was above T's maximum (30,423); in the editorial plan it was below T's minimum (442,252 against 468,129). With the strategy, the values the prompts prescribe (result limits, search depth, keywords per call) were respected in all 30 runs of the two cases whose prompts prescribe them. Runs that asked a question instead of doing the task: one of the 25 with the strategy (in arm ST), no of those without it; that run is kept in every figure. In the billing audit (Section 7.2) the period-end ticket failed in two runs, all with the strategy text.

C.2 E3: the strategy text on a second model, without trimming

E1 left open what the strategy does for a model that issues calls one at a time. E3 asked it of `gemini-3.8-flash`, with arms R and S and no trimming, on billing support and CI triage, whose simulated servers were live, and on the editorial plan and the content-gap analysis, whose search-data provider was replaced by recorded outputs: a call got the recorded output of the same call when there was one and otherwise the output of the recorded call of the same tool with the most similar arguments. The provider was called in no run. On those two cases only the organisation of calls is evaluated, not the answers. Runs per arm go from two to eight (Table 15).

The hypotheses on turns and grouped calls failed as stated. P4-H8 (median turns with the strategy below the fewest turns of any run without it) held in two cases of four, the two with recorded outputs; P4-H9 (median share of turns with several calls above the highest share of any run without it) held in no case: in each, a run without the strategy reached the same share or more. Why they failed concerns the rule, not the runs, whose results stand as they are. Comparing the median of one arm with the extreme of the other lets a single run decide: with six runs per arm, as on CI triage, one unusual run without the strategy is enough to fail the hypothesis, and there that is what happened. A protocol that wants to know whether an arm is usually shorter should compare what is usual in both. The two simulated cases are therefore read apart.

CI triage, where this model issued one call per turn. Without the strategy, five runs of six made every call in a turn of its own (14 to 19 turns); the remaining one grouped its calls and took 5. With the strategy all runs took 5 to 8 turns, with a similar number of calls. The median went from 17 turns to 6, billed input tokens fell by 55%, and the LLM API cost per run by 15%: less than the tokens, because most of the input re-read at each turn is billed at the cached price. P4-H8 fails on this case because of that one run: the strategy's median, 6, is not below 5.

Billing support, where it already grouped its calls. Little was left: runs without the strategy took 3 to 8 turns, those with it always 3, which is the fewest this task needs (see the post-hoc analysis below).

The implemented gates and coverage checks detected no loss, and the prompts were not overridden. P4-H10 holds: the gates passed in 8/8 and 6/6 runs of both arms of billing support and CI triage, every requirement of the prompts covered, as far as the gates and the coverage checks test them. P4-H11 holds: the prescribed values were respected in 9 of 9 checks, in the five strategy runs with recorded outputs. Those two cases point the same way as CI triage (median turns from 14 to 10.5 in the editorial plan, from 6 to 5 in the content-gap analysis), on two and three runs per arm and with recorded outputs: the model's call matched a recorded call exactly in 11% to 40% of the calls of a run in the editorial plan and in 5% to 8% in the content-gap analysis, and got the nearest recorded output otherwise. They say how calls were organised, not whether the task was done well.

Post hoc: calls that could have been made earlier. This analysis is not in the protocol; it was added after the runs, to tell a sequence the task imposes from one the model chose. A call whose arguments name something an earlier result returned (a run, a job, a customer, an invoice) can only follow that result; any other could have been made at once. The longest chain of such calls gives the fewest turns a run needed. In CI triage 72% of the calls depended on an earlier result, but the chain was short (the runs, then their jobs, then the logs): four turns were enough in every run. Without the strategy 77% of the calls were made later than they could have been, with it 44%; in billing support 19% and 0%, with three turns needed. The rule reads identifiers only and cannot know that a model wanted to read one log before asking for the next: it gives a lower bound on the turns, not what a careful agent should do.

C.3 The hosts checked

After the runs of Study 3, the proxy was put in front of servers in several hosts, in most checks with a log that records each request of the host's client and none of its content; the repository's page on hosts states what was verified, when and how, and everything else as not verified. In the OpenAI Playground, Claude Code (on Linux and on Windows) and VS Code (Copilot Chat in agent mode), the tools passed and the model called `tracetrim_done` at the end of its run; in the Playground it did so guided by the tool's description alone. `expand` was verified in the Playground only: in the other hosts nothing had yet been trimmed. MCP Inspector, which has no model, showed that the skill is served as the MCP Skills extension asks. None of the hosts with a model loaded the skill as a skill: no client declared the extension. The Playground's and VS Code's clients asked for neither the skill nor resources; in Claude Code the model read the skill through the host's tool for MCP resources before its first call, as a resource. Claude Desktop and Cursor were not verified. These checks show that the path works in those hosts, not how often or how well.